UNIVERSITY OF WEST ATTICA
FACULTY OF ENGINEERING
Department of Electrical & Electronics Engineering
Department of Industrial Design & Production Engineering
MSc in Artificial Intelligence and Deep Learning

**Master of Science Thesis**

# Development of estimation algorithms for navigation of unmanned aerial vehicles



**Student: Kostakis Ilias-Odysseas**
**Registration Number: AIDL-0063**

**MSc Thesis Supervisor**

**Alexandridis Alex,**
**Professor**

**ATHENS-EGALEO, June 2026**

This MSc Thesis has been accepted, evaluated and graded by the following committee:

| Supervisor | Member | Member |
|---|---|---|
| Digitally signed by ALEXANDROS ALEXANDRIDIS Date: 2026.07.13 12:08:08 +03'00' | Digitally signed by PANAGIOTIS PAPAGEORGAS Date: 2026.07.13 08:19:03 +03'00' | ILIAS ZOIS Digitally signed by ILIAS ZOIS Date: 2026.07.13 10:24:31 +03'00' |
| Alexandridis, Alex | Papageorgas, Panagiotis | Zois, Elias |
| Professor | Professor | Associate Professor |
| Department of Electrical and Electronic Engineering | Department of Electrical and Electronic Engineering | Department of Electrical and Electronic Engineering |
| University of West Attica | University of West Attica | University of West Attica |

**Copyright ©** All rights reserved.

**University of West Attica and Ilias-Odysseas Kostakis**
**June, 2026**

You may not copy, reproduce or distribute this work (or any part of it) for commercial purposes. Copying/reprinting, storage and distribution for any non-profit educational or research purposes are allowed under the conditions of referring to the original source and of reproducing the present copyright note. Any inquiries relevant to the use of this thesis for profit/commercial purposes must be addressed to the author.

The opinions and the conclusions included in this document express solely the author and do not express the opinion of the MSc thesis supervisor or the examination committee or the formal position of the Department(s) or the University of West Attica.

**Declaration of the author of this MSc thesis**

I, Ilias-Odysseas Kostakis with the following student registration number aidl-0063, postgraduate student of the MSc program in "Artificial Intelligence and Deep Learning", which is organized by the Department of Electrical and Electronic Engineering and the Department of Industrial Design and Production Engineering of the Faculty of Engineering of the University of West Attica, hereby declare that:

I am the author of this MSc thesis and any help I may have received is clearly mentioned in the thesis. Additionally, all the sources I have used (e.g., to extract data, ideas, words or phrases) are cited with full reference to the corresponding authors, the publishing house or the journal; this also applies to the Internet sources that I have used. I also confirm that I have personally written this thesis and the intellectual property rights belong to myself and to the University of West Attica. This work has not been submitted for any other degree or professional qualification except as specified in it.

Any violations of my academic responsibilities, as stated above, constitutes substantial reason for the cancellation of the conferred MSc degree.

The author

Ilias-Odysseas Kostakis

## Abstract

Reliable and accurate attitude estimation and prediction have a crucial role for the autonomous navigation and control of unmanned aerial vehicles (UAVs). Classic sensor fusion algorithms are designed to estimate the current UAV orientation and cannot predict or anticipate possible future states of the vehicle. Technologies like Model Predictive Control (MPC) require multi-step ahead predictions of future attitude. This creates a need for predictive models. This thesis develops a UAV attitude prediction framework using a Long Short-Term Memory (LSTM) network and explores whether an attention-based mechanism can be used. The architecture is designed to predict future roll $\phi$, pitch $\theta$, and yaw $\psi$ angles over a configurable multi-step horizon using look-back windows of IMU measurements, control inputs and previously estimated attitude angles. In order to provide the most correct estimated angles four classic tilt estimation filters were implemented: Complementary Filter, Mahony Filter, Explicit Complementary Filter, and Extended Kalman Filter (EKF). The prediction framework uses training data that were labeled from an external ground-truth system. In a real deployment scenario it operates individually by accepting EKF-estimated angles as input. Experiments have shown that the LSTM model achieves both high estimation accuracy and provides the essential multi-step prediction capability that is required for the MPC based flight controllers.

## Keywords

Attitude estimation, Extended Kalman Filter, Long Short-Term Memory, Machine Learning, Neural Networks, Recurrent Neural Networks, Sensor Fusion, Unmanned Aerial Vehicle

## Table of Contents

**List of Tables** ..... 6

**List of figures** ..... 6

**Acronym Index** ..... 7

**1 Chapter 1: Introduction** ..... 8

**2 Chapter 2: Theoretical Background** ..... 10

**2.1 The role of attitude in UAV flight** ..... 10

**2.2 IMU and Sensors** ..... 10

2.2.1 Gyroscope ..... 10

2.2.2 Accelerometer ..... 11

2.2.3 Quadcopter Attitude Dynamics ..... 11

**2.3 Attitude Representation** ..... 12

2.3.1 Euler Angles ..... 12

2.3.2 Rotation Matrices (SO(3)) ..... 12

2.3.3 Quaternions ..... 12

2.3.4 Sensor Fusion Fundamentals ..... 13

**2.4 Neural Networks for Time-Series Predictions** ..... 13

2.4.1 Time-Series Data and Sequential Learning ..... 13

2.4.2 Recurrent Neural Networks (RNN) ..... 14

2.4.3 Long Short-Term Memory (LSTM) ..... 14

2.4.4 Attention Mechanisms ..... 15

**3 Chapter 3: Tilt Estimation Filters** ..... 15

**3.1 Quadcopter and Dataset** ..... 16

**3.2 Complementary Filtering** ..... 16

3.2.1 Algorithm Overview ..... 16

3.2.2 Parameters ..... 16

3.2.3 Experimental results ..... 17

**3.3 Mahony Filter** ..... 19

3.3.1 Algorithm Overview ..... 19

3.3.2 Parameters ..... 19

3.3.3 Experimental Results ..... 20

**3.4 Explicit Complementary Filtering** ..... 22

3.4.1 Algorithm Overview ..... 22

3.4.2 Parameters ..... 23

3.4.3 Experimental Results ..... 23

**3.5 Extended Kalman Filtering (EKF)** ..... 25

3.5.1 Algorithm Overview ..... 25

3.5.2 Parameters ..... 26

3.5.3 Experimental Results ..... 27

**3.6 Limitations and Conclusions** ..... 29

**4 Chapter 4: Deep Learning Approach** ..... 30

**4.1 Problem Formulation** ..... 31

4.1.1 Prediction Task Definition ..... 31

4.1.2 Input Feature Vector ..... 31

4.1.3 Output Representation ..... 32
4.1.4 Dataset ..... 32
**4.2 Network Architecture ..... 32**
4.2.1 LSTM Layer ..... 33
4.2.2 Attention Mechanism ..... 34
4.2.3 Dropout Regularization ..... 34
4.2.4 Output Layer ..... 35
**4.3 Training Methodology ..... 35**
4.3.1 Sliding Windows ..... 35
4.3.2 Data Standardization ..... 35
4.3.3 Data Splitting ..... 36
4.3.4 Loss & Optimization ..... 36
**4.4 Experimental Results and Evaluation ..... 37**
4.4.1 Overall Performance Comparison ..... 37
4.4.2 Error Growth Over the Prediction Horizon ..... 40
**4.5 Discussion ..... 41**
4.5.1 The role of Attention ..... 41
4.5.2 Limitations ..... 41
**5 Conclusions ..... 41**
**6 Future Work ..... 42**
**Bibliography – References – Online sources ..... 42**

## List of Tables

Table 1 Summary of Classical Filter Performance ..... 30
Table 2 Data split percentage ..... 36
Table 3 Training Configuration ..... 37
Table 4 LSTM Model's Definition ..... 37
Table 5 LSTM Model's RMSE ..... 38

## List of figures

Figure 1 Complementary Filter Roll and Pitch Estimation ..... 17
Figure 2 Complementary Filter Zoomed view (samples 300-500) Pitch Estimation ..... 18
Figure 3 Complementary Filter Zoomed view (samples 300-500) Roll Estimation ..... 18
Figure 4 Mahony Filter Roll and Pitch Estimation ..... 20
Figure 5 Mahony Filter Zoomed view (samples 300-500) Pitch Estimation ..... 21
Figure 6 Mahony Filter Zoomed view (samples 300-500) Roll Estimation ..... 21

Figure 7 Explicit Complementary Filter Roll and Pitch Estimation ..... 23

Figure 8 Explicit Complementary Zoomed view (samples 300-500) Pitch Estimation ..... 24

Figure 9 Explicit Complementary Zoomed view (samples 300-500) Roll Estimation ..... 24

Figure 10 EKF Roll and Pitch Estimation ..... 27

Figure 11 EKF Zoomed view (samples 300-500) Pitch Estimation ..... 28

Figure 12 EKF Zoomed view (samples 300-500) Roll Estimation ..... 28

Figure 13 Filters comparison in specific window ..... 30

Figure 14 LSTM Attention Based Network Architecture for K=10 ..... 33

Figure 15 LSTM Model Comparison Test RMSE ..... 39

Figure 16 Model E 30 steps prediction ..... 39

Figure 17 Model E Error Growth ..... 40

## Acronym Index

CNN: Convolutional Neural Network
DOF: Degrees of Freedom
EKF: Extended Kalman Filter
GNSS: Global Navigation Satellite System
GRU: Gated Recurrent Unit
IMU: Inertial Measurement Unit
LSTM: Long Short-Term Memory
MAE: Mean Absolute Error
MPC: Model Predictive Control
MSE: Mean Squared Error
PID: Proportional-Integral-Derivative
RMSE: Root Mean Square Error
RNN: Recurrent Neural Network
SO(3): Special Orthogonal Group in 3 Dimensions
SVD: Singular Value Decomposition
UAV: Unmanned Aerial Vehicle

# 1 Chapter 1: Introduction

Unmanned Aerial Vehicles (UAVs) have been an evolving technology over recent years, from aerial photographs to package delivery. The most critical capability of a UAV is to know its orientation in three-dimensional space [1], [8]. Without continuous and accurate attitude information, a UAV cannot keep steady flight or respond to disturbances. Modern UAVs rely on Inertial Measurement Units (IMUs) and sensor fusion algorithms. These algorithms are based on gyroscope and accelerometer sensors. Because individual sensors have limitations, classical sensor fusion algorithms like the Extended Kalman Filter (EKF) are used to produce reliable estimates of the current attitude. However, while effective for real-time estimation, these approaches are fundamentally reactive - they can only estimate the present state but cannot anticipate the future.

This limitation becomes significant when we consider how modern flight controllers actually operate. Proactive control strategies, such as Model Predictive Control (MPC) [2]–[5] or backstepping control [6], do not react to the current error. At each step they solve an optimization problem over a finite future horizon [7]. Rather than reacting to the current error, MPC evaluates predicted future trajectories to compute a sequence of optimal control actions, applying only the first action before re-solving with updated measurements at the next timestep. This receding-horizon methodology allows the controller to anticipate upcoming disturbances before they happen. It is clear that this process depends on a prediction model that is capable of supplying accurate enough attitude estimation angles over that horizon. This is a requirement classical reactive sensor fusion filters cannot satisfy.

Deep learning has been studied as a response to the above limitation. For instance, Liu et al. [9] demonstrated LSTM-based orientation estimation for UAVs, showing that recurrent networks can learn temporal patterns in IMU data. Weber et al. [10] proposed RIANN, a GRU-based network that consistently outperformed classical filters across different motion types without dataset tuning. Golroudbari and Sabour [11] developed hybrid CNN-LSTM architectures, that achieved very high accuracy for real-time attitude estimation, while Brossard et al. [12] and Cohen and Klein [13] investigated convolutional and hybrid models that reduce noise from the sensors. Furthermore, Bianchi et al. [14] explored physics-informed neural networks for UAV system estimation by incorporating known flight dynamics into the learning process.

A common limitation runs through all of these deep learning approaches: they focus mostly on the single-step estimation of the current attitude state. While single-step estimation is the standard benchmark in these works, it does not address the need to forecast future attitude angles over a prediction horizon of several timesteps. This difference must be highlighted. In most cases accurate current-state estimation can be sufficient, but it cannot be used for proactive control strategies such as MPC since they require predictions of future states to compute optimal control actions[2]. This creates a need for predictive models that can provide the extended, multi-step prediction horizons inherently required by MPC-based flight controllers.

This identified gap is what this thesis addresses. The task is to predict future UAV attitude using past sensor data and control data. The main challenge is that true attitude cannot be measured

directly from a single sensor. Initially, it has to be estimated from raw IMU data and then it can be given as input to the LSTM model to predict ahead. It is obvious that the quality of the estimate has an impact on the prediction quality, so these two stages cannot be separated. They are being treated as one pipeline, classical sensor fusion to produce the attitude and deep learning to make the prediction.

The main goal is to design, implement, and evaluate a complete estimation to prediction pipeline suitable for real-time deployment on UAV hardware. Objectives can be separated as follows. Firstly, implement and compare four classical tilt estimation filters as a performance baseline. Second, to design and train an LSTM-based architecture with an optional attention mechanism for multi-step attitude prediction. Next evaluate prediction accuracy across multiple steps.

The methodology is as follows: The first stage, processes raw 6-DOF IMU measurements using an Extended Kalman Filter (EKF), which ensures an accurate estimate of the current attitude. In the second stage, a predictive deep learning model analyses a sliding window of recent flight data. The input sequence combines three complementary groups of features: the current attitude estimated by the EKF (representing the spatial state), the raw gyroscope angular velocities (capturing the rate of change), and the active PID control torques (representing the control inputs). Rather than compressing the historical information into a single hidden state, as it is common in conventional recurrent architectures, the proposed model incorporates an attention mechanism that dynamically assigns importance to individual timesteps within the input sequence. This enables the network to emphasize the most informative parts of the flight history, particularly during abrupt control actions or rapid maneuvers. The model then produces simultaneous multi-step predictions of roll, pitch, and yaw through a Direct Multi-Output regression strategy, satisfying the prediction requirements of Model Predictive Control (MPC).

The main contribution of this work is the focus on predictive attitude modelling rather than attitude estimation alone. While previous studies have largely explored single-step attitude estimation as an alternative to classical filtering techniques, the proposed framework is designed specifically for multi-step attitude prediction, allowing the flight controller to anticipate future system states instead of responding only to current measurements.

The proposed framework has also been designed with practical deployment in mind. Although the neural network is initially trained using labelled flight data obtained from an external ground-truth system, no external infrastructure is required once the model is deployed. During operation, an Extended Kalman Filter estimates the current attitude in real time. Also, the entire deep learning framework has been implemented in C++ using LibTorch and Eigen rather than relying on Python-based environments. This implementation facilitates direct deployment on resource-constrained embedded UAV platforms, bridging the gap between algorithm development and real-world application.

The rest of the thesis is structured as follows. In chapter 2, the necessary theory is introduced, namely the basics of UAV attitude representation, inertial sensors and sequential learning based on recurrent neural networks. Chapter 3 details the implementation, parameter tuning and comparison of four classical tilt estimation filters (Complementary, Mahony, Explicit

Complementary, EKF) on real flight data. The main contribution of the work is done in chapter 4, where the attention-based LSTM architecture is designed, trained and evaluated to predict UAV attitude multi-steps ahead. In the last section of the thesis, conclusions are drawn and future work is presented.

# 2 Chapter 2: Theoretical Background

This section explains the basic terms used in UAV attitude estimation. It starts with the role of attitude in flight dynamics and explains the inertial sensors that measure motion. Then it looks at various ways an orientation can be expressed mathematically. It continues by explaining the concepts of sensor fusion which is the basis of Chapter 3. The last part covers the sequential learning with neural networks, which is the background for deep learning approach used in Chapter 4.

## 2.1 The role of attitude in UAV flight

The autonomous operation of drones depends heavily on their ability to determine and control their orientation in three-dimensional space. The attitude of a UAV describes its orientation relative to a fixed reference frame, typically defined with respect to the Earth's surface. In aviation attitude is mostly described using three angular coordinates known as Euler Angles. These angles directly affect the vehicle's ability to maintain stable flight, do maneuvers, and respond to various disturbances. Without accurate attitude information, a UAV cannot maintain stable flight, execute controlled turns and implement autonomous navigation algorithms. Attitude is harder to obtain compared to position or velocity. GPS can measure position directly, and velocity can be retrieved from multiple successive GPS readings, but no single sensor can measure attitude. Attitude is not something a sensor reads off directly. It has to be pieced together from multiple sensors, and each of those sensors has its own particular weaknesses to work around.

## 2.2 IMU and Sensors

An inertial measurement unit (IMU) is an electronic device that measures and reports a body's specific force, angular rate and sometimes the orientation of the body, using a combination of accelerators, gyroscopes and sometimes magnetometers. When the magnetometer is included, IMUs are referred to as IMMUs.

### 2.2.1 Gyroscope

A gyroscope is a device used for measuring or maintaining orientation and angular velocity. It's main purpose when used in a UAV is to measure the rotation speed. The speed is usually measured in $rad/s$ across all the sensor's axis (x, y, z). Gyroscopes are not affected by linear accelerations. This makes them reliable during aggressive maneuverers or sudden changes of speed. The main drawback is that in the long term they suffer from noise and bias. This is because the UAVs have mechanical sensors (MEMS) which are highly sensitive to engine vibrations, temperature changes, and minor electronic interference, making them lose accuracy over time. Many techniques have been developed to address this. The common one is to use

data from accelerometers, magnetometers, or GNSS to constantly pull the gyroscope's estimates back to reality [15].

### 2.2.2 Accelerometer

Rather than measuring purely kinematic acceleration, an accelerometer measures "specific force" or proper acceleration ($\alpha \in R^3$) in $m/s^2$. Because it continuously senses the acceleration due to gravity, it provides an excellent long-term absolute reference frame. When the UAV is stationary or moving slowly, this gravity vector allows the system to accurately calculate the vehicle's roll ($\phi$) and pitch ($\theta$) angles using trigonometry.

$$\phi = atan2(a_y, a_z)$$
$$\theta = arsin\left(\frac{a_x}{g}\right) \quad (1)$$

### 2.2.3 Quadcopter Attitude Dynamics

The attitude dynamics of a quadcopter are governed by two coupled sets of equations: the kinematic equations, which relate the time derivatives of the Euler angles to the body-frame angular velocities, and the dynamic equations, which describe how those angular velocities evolve under applied torques.

Under the assumption of small angles, the kinematic relations simplify to a direct correspondence between the Euler angle rates and the body angular velocities:

$$\dot{\phi} = \omega_1, \dot{\theta} = \omega_2, \dot{\psi} = \omega_3 \quad (2)$$

where $\phi, \theta, \psi$ are the roll, pitch, and yaw angles and $\omega_1, \omega_2, \omega_3$ are the corresponding angular velocities measured in body frame.

The evolution of these angular velocities is governed by Euler's equations for rigid-body rotation, extended to account for the gyroscopic effect of the spinning propellers [14].

$$\dot{\omega_1}(t) = \frac{I_2 - I_3}{I_1}\omega_2(t)\omega_3(t) + \frac{I_p}{I_1}\omega_p(t)\omega_2(t) + \frac{1}{I_1}\tau_1(t) \quad (3)$$

$$\dot{\omega_2}(t) = \frac{I_3 - I_1}{I_2}\omega_1(t)\omega_3(t) - \frac{I_p}{I_2}\omega_p(t)\omega_1(t) + \frac{1}{I_2}\tau_2(t) \quad (4)$$

$$\dot{\omega_3}(t) = \frac{I_1 - I_2}{I_3}\omega_1(t)\omega_2(t) + \frac{1}{I_3}\tau_3(t) \quad (5)$$

where $I_1, I_2, I_3$ are the principal moments of inertia about the roll, pitch, and yaw axes respectively, $I_p$ is the moment of inertia of the propeller assembly, $\omega_p$ is the net propeller angular speed (the algebraic sum of all rotor speeds), and $\tau_1, \tau_2, \tau_3$ are the control torques generated by the differential rotor thrust. The cross-product terms (e.g. $\omega_2\omega_3$) represent the Coriolis coupling between axes inherent to rotating frames, while the $I_p$ terms capture the gyroscopic precession effect of the propellers.

## 2.3 Attitude Representation

An attitude representation is often defined as a set of coordinates that describe the orientation of a given reference frame with respect to a second reference frame. Defining the rotational orientation of a rigid body requires a minimum of three parameters, however, many attitude representations utilize more than three parameters. While there are numerous attitude representations that can be used to define the orientation, some of the more common representations include, Euler angles, rotation matrices, and quaternions.

### 2.3.1 Euler Angles

Euler angles offer the easiest way to visualize orientation by breaking down an object's attitude into three sequential rotations. In aerospace applications, the standard approach is the ZYX (yaw-pitch-roll) sequence [16], which applies rotations in this specific order:

1. $\psi$ (yaw) around the vertical Z-axis.
2. $\theta$ (pitch) around the Y-axis.
3. $\phi$ (roll) around the X-axis.

Because this method only relies on three parameters, it is incredibly easy for humans to read and requires minimal memory to compute. The biggest drawback is that the angles are not unique. This means that different combinations of angles can result in the same physical orientation. Also converting from angle rates and angular velocity is a trigonometry math operation which is costly in terms of computation. Despite these drawbacks each filter in this thesis converts each results back to Euler angles. This is done because it is the most practical format when interpreting data and showing graphs with comparisons to truth data.

### 2.3.2 Rotation Matrices (SO(3))

A rotation matrix $R \in SO(3)$ is a 3×3 orthonormal matrix that transforms vectors from one coordinate frame to another. The notation $SO(3)$ stands for "Special Orthogonal group in 3 dimensions," representing all possible rotations in three-dimensional space. A rotation matrix can be constructed from Euler angles using the product of elementary rotation matrices [17]:

$$\boldsymbol{R(\varphi, \theta, \psi) = R_z(\psi) \cdot R_y(\theta) \cdot R_x(\varphi)} \quad (6)$$

where $R_x, R_y, R_z$ represent rotations around the X, Y, and Z axes respectively. The 3×3 matrix has nine elements, but only three are independent because of the orthonormality constraints, and this is the main drawback of rotation matrices. Storing nine numbers for three degrees is not efficient. Also, the matrices require periodic orthonormalization to correct numerical drift. The most used methods for this are Gram-Schmidt and singular value decomposition (SVD) [17].

### 2.3.3 Quaternions

To get around the inefficiencies of rotation matrices, we can use quaternions. A quaternion denoted as $q$, describes a rotation using just four parameters rather than nine. It breaks down into a single scalar component $q_0$ and a 3D vector component $q_v$ [16].

$$q = [q_0, q_1, q_2, q_3]^T = [q_0, q_v]^T \quad (7)$$

Just like the matrices discussed earlier, quaternions still suffer from numerical drift during integration and must be periodically normalized to preserve their unit length constraint. In this case though, the computational cost is much lower. Re-normalizing a quaternion only requires a simple division operation, making it much cheaper to run compared to a full matrix orthonormalization projection.

### 2.3.4 Sensor Fusion Fundamentals

Sensor fusion combines several sensors to get a better estimate than any of them could give alone. Gyroscope and accelerometer work well together because their errors are the opposite. A gyroscope gives smooth and fast measurements but drifts because integration accumulates error. On the other hand, an accelerometer does not drift but it is noisy and gets thrown off by linear acceleration.

A fusion algorithm must do many things at once. First, gather gyroscope measurements for short-term movement. Then use the accelerometer to remove the drift and lastly ignore the accelerometer when the vehicle is accelerating and its reading can not be trusted.

The filters in this thesis cover various approaches, from simple weighted average (complementary filter) to nonlinear filters on the $SO(3)$, to full probabilistic estimation (EKF). All of them are scored against ground truth using Root Mean Square Error (RMSE) and Mean Absolute Error (MAE).

## 2.4 Neural Networks for Time-Series Predictions

As it is discussed the classical filters rely on mathematical models. Deep learning takes the opposite approach: it learns behavior directly from data. This section builds up the background for the LSTM network that gets developed in Chapter 4. Covers sequential learning, recurrent networks and the attention mechanism that multi-step prediction rely on.

### 2.4.1 Time-Series Data and Sequential Learning

Attitude estimation is fundamentally a time-series problem. IMU measurements arrive sequentially at fixed sampling intervals, denoted as $T_s$, and the vehicle's state at a given time step $t$ is highly dependent on its state at $t-1$. Feedforward networks can be used to use the current state and predict the next one, but they were not built for this purpose. There is no internal memory to capture the long and complex sequences. Each prediction is made in isolation, as if the network has never seen the sequence before. This is a problem for UAVs because they need to distinguish the difference between the maneuverer and the sensor noise. To do this they need to look across several consecutive readings not just one.

To understand this in a mathematical way, this is how a standard feedforward layer computes the output for a single layer:

$$\hat{y} = W_2 \cdot \sigma(W_1 \cdot x_t + b_1) + b_2 \tag{8}$$

where $x_t \in \mathbb{R}^m$ is the input vector, $W_1, W_2$ are weight matrices, $b_1, b_2$ are bias vectors, $\sigma(\cdot)$ is a nonlinear activation function, and $\hat{y}$ is the predicted output [21]. We can see that $\hat{y}$ depends only

on $x_t$, with no term carrying any past input (e.g. $x_{t-1}$). Any kind of important past information is lost.

### 2.4.2 Recurrent Neural Networks (RNN)

RNNs address the limitations of feedforward networks by maintaining a hidden state $h_t$ that acts as a memory of previous inputs. At each timestep $t$, RNN processes the current input $x_t$ together with the accumulated context of previous hidden states, which is recursively passed forward from the immediately preceding state $h_{t-1}$. While theoretically capable of learning long sequences, standard RNNs suffer from the vanishing gradient problem, where information from earlier timesteps is lost during training [18]. For attitude prediction tasks which usually require a big history window standard RNNs cannot be used. At each timestep $t$ the RNN computes:

$$\begin{aligned} h_t &= \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b_h) \\ \hat{y}_t &= W_o \cdot h_t + b_o \end{aligned} \tag{9}$$

where $h_t \in \mathbb{R}^{\mathbb{d}}$ is the hidden state at timestep $t$, $x_t$ is the current input. $W_h, W_x, W_o$, are learnable weight matrices, and $b_h$, $b_o$ are bias vectors [21].

### 2.4.3 Long Short-Term Memory (LSTM)

The LSTM architecture that was introduced by Hochreiter and Schmidhuber [19], solves the vanishing gradient problem through a complex gating mechanism. It maintains a cell state $C_t$ that carries information across long intervals. The flow of information is controlled by three gates:

**Forget Gate**: Determines which historical data to discard:

$$f_t = \sigma\big(W_f \cdot [h_{t-1}, x_t] + b_f\big) \tag{10}$$

**Input Gate**: Updates the cell state with new information

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{11}$$

**Output Gate**: Filter the cell state to produce the hidden state

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{12}$$

For IMU data, LSTMs can learn to ignore high-frequency vibration noise (via the forget gate) while focusing on the persistent gravity vector (via the input gate). The cell state is updated as:

$$\begin{aligned} \widetilde{C_t} &= \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \\ C_t &= f_t \odot C_{t-1} + i_t \odot \widetilde{C_t} \\ h_t &= o_t \odot \tanh(C_t) \end{aligned} \tag{13}$$

where σ is the sigmoid function, $\odot$ is the element-wise (Hadamard) product, and $[h_{t-1}, x_t]$ denotes vector concatenation. The forget gate $f_t$ selectively discards past cell state, the input gate $i_t$ controls how much new information enters, and the output gate $o_t$ filters the cell state to produce the hidden state $h_t$.

### 2.4.4 Attention Mechanisms

Standard LSTMs use only the final hidden state $h_k$ to generate predictions, which can create an information bottleneck. To improve prediction accuracy over an N-step horizon this work incorporates an **Attention Mechanism**. This layer assigns a score for every hidden state in the look-back window, effectively allowing the model to pay attention to past events (e.g., a sudden change in $\omega_{gyro}$) that are most relevant for making a predicting about the future state. The final context vector c is a weighted sum:

$$c = \sum_{i=1}^{K} a_i h_i \tag{14}$$

where $a_i$ represents the learned importance of the timestep $i$. The weights $a_i$ are computed by passing each hidden state through a scoring function followed by a softmax normalization, ensuring they sum to one. In this way, the network can learn to attend to different parts of the input sequence, depending on the prediction being made.

## 3 Chapter 3: Tilt Estimation Filters

This chapter presents the implementation, tuning, and analysis of four classical sensor fusion algorithms for attitude estimation in UAVs. Building on the theoretical developments of Chapter 2, we demonstrate how the different mathematical formulations are related to practical filtering approaches with tradeoffs in computational complexity and estimation performance.

The four filters used in this work increase in mathematical complexity. Complementary filters provide a baseline implementation of a weighted fusion of gyroscope and accelerometer measurements. The Mahony Passive Complementary Filter extends this concept by operating on the $SO(3)$ space. The explicit complementary filter take the Mahony algorithm and extended it to incorporate integral feedback for gyroscope bias estimation. The Extended Kalman Filter (EKF) implements a full probabilistic state estimation framework using quaternion representation, handling nonlinear dynamics through local linearization under Gaussian noise assumptions.

All implementations are developed in C++ using the Eigen library for all the mathematical operations. Each of the filters is given an input of 3-axis gyroscope and accelerometer measurements from a 6-DOF IMU operating at 50 Hz sampling rate. They are tested against ground truth from a flight recording of 1409 samples, covering everything from hover to aggressive maneuvers.

In this chapter, for each filter, we give the mathematical expression, the implementation method, the hyperparameters tuning and the performance of the RMSE and MAE for the roll and pitch estimation.

## 3.1 Quadcopter and Dataset

The flight data used throughout this study, both for evaluating the classical filtering algorithms and for training the LSTM model, were collected using a custom-built quadcopter based on an F450 frame. The platform is equipped with 1100 kV brushless outrunner motors, 8045 propellers, and 30 A electronic speed controllers, powered by a 2700 mAh LiPo battery. The onboard avionics consist of a Navio2 autopilot integrated with a Raspberry Pi 4 Model B. Inertial measurement is handled by dual IMUs — the MPU9250 and LSM9DS1 — each offering three-axis gyroscope, accelerometer, and magnetometer outputs.

Data were collected during real flight operations, in which a PID controller was employed to ensure stable flight conditions and precise tracking across all three quadcopter orientation angles (roll, pitch and yaw).

## 3.2 Complementary Filtering

The complementary filter is a quick and effective way for combining measurements from accelerometer and gyroscope to generate an estimate for orientation [20]. Because it was a very low computational cost it is used a lot in low-cost UAV applications.

### 3.2.1 Algorithm Overview

The algorithm is depended on the fact that the gyroscope and accelerometer expose complementary spectral errors. As described earlier gyroscopes provide precise and low-noise measurements of angular velocity for short term but they suffer from drift when used in the long term. On the other hand, accelerometers provide stable reference to the gravity vector that is reliable in the long term but they suffer from noise in external linear acceleration during dynamic maneuvers. The discrete-time implementation for the roll angle $\phi$ is given by:

$$\varphi(t) = \alpha \cdot \left(\phi(t-1) + \omega_{gyro} \cdot dt\right) + (1-\alpha) \cdot \phi_{accel} \tag{15}$$

### 3.2.2 Parameters

The filter has only one parameter $\alpha$ that controls how much weight to put on each sensor data. It takes values in the range of (0,1). Value close to zero means to put more trust in the accelerometer. Values closer to one means to put more trust in the gyroscope data.

To determine the optimal value, a grid search was performed over $\alpha \in [0.01, 0.99]$ with a step size of 0.01. For each value, the filter was executed on the full dataset and the RMSE of the $\phi$ estimation against ground truth was computed. The search identified $\alpha = 0.79$ as the value minimizing the roll RMSE.

### 3.2.3 Experimental results

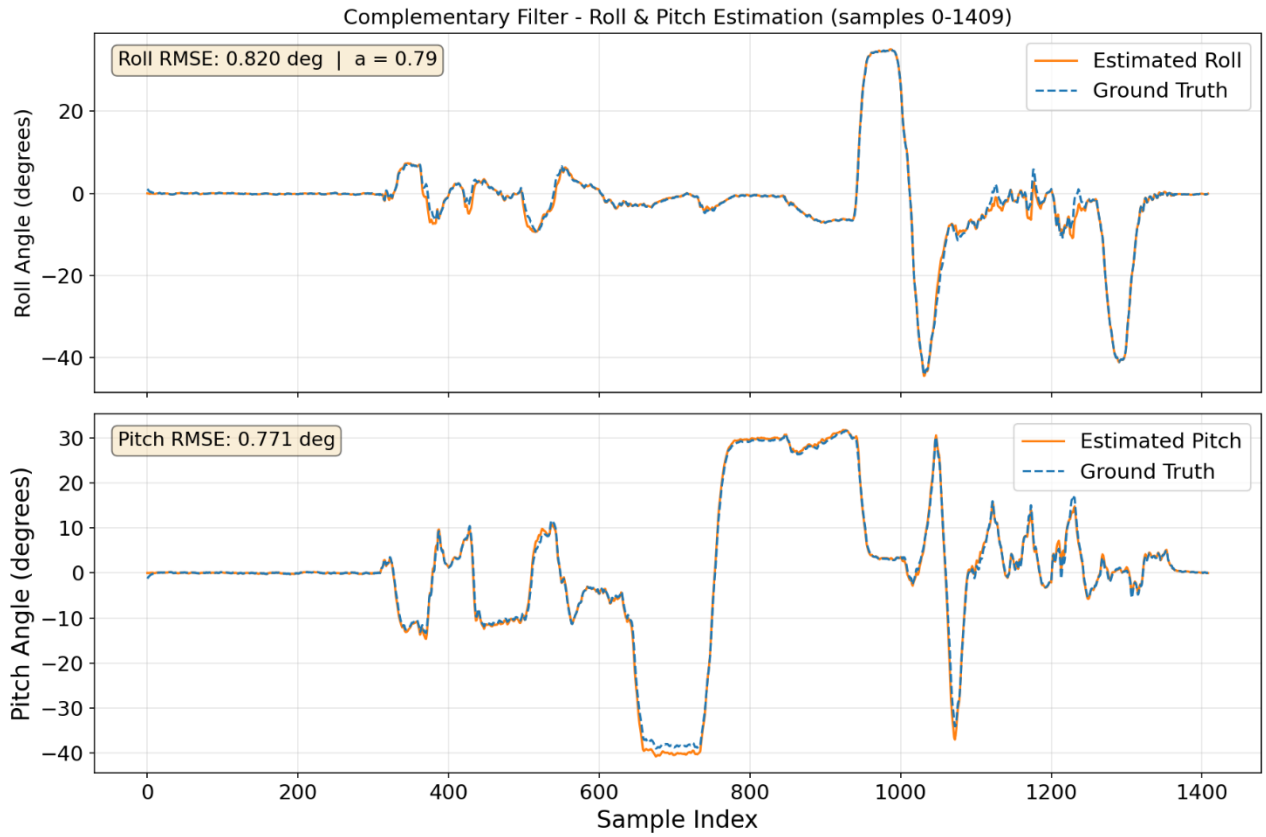



**Figure 1 Complementary Filter Roll and Pitch Estimation**

Figure 1 shows the global performance of the Complementary Filter throughout the flight (1409 samples). As can be observed, the estimated Roll and Pitch angles correlate well with the ground truth. The small RMSE values 0.820° for Roll and 0.771° for Pitch verify that the filter has correctly combined high-frequency gyroscope information with the low-frequency accelerometer information.

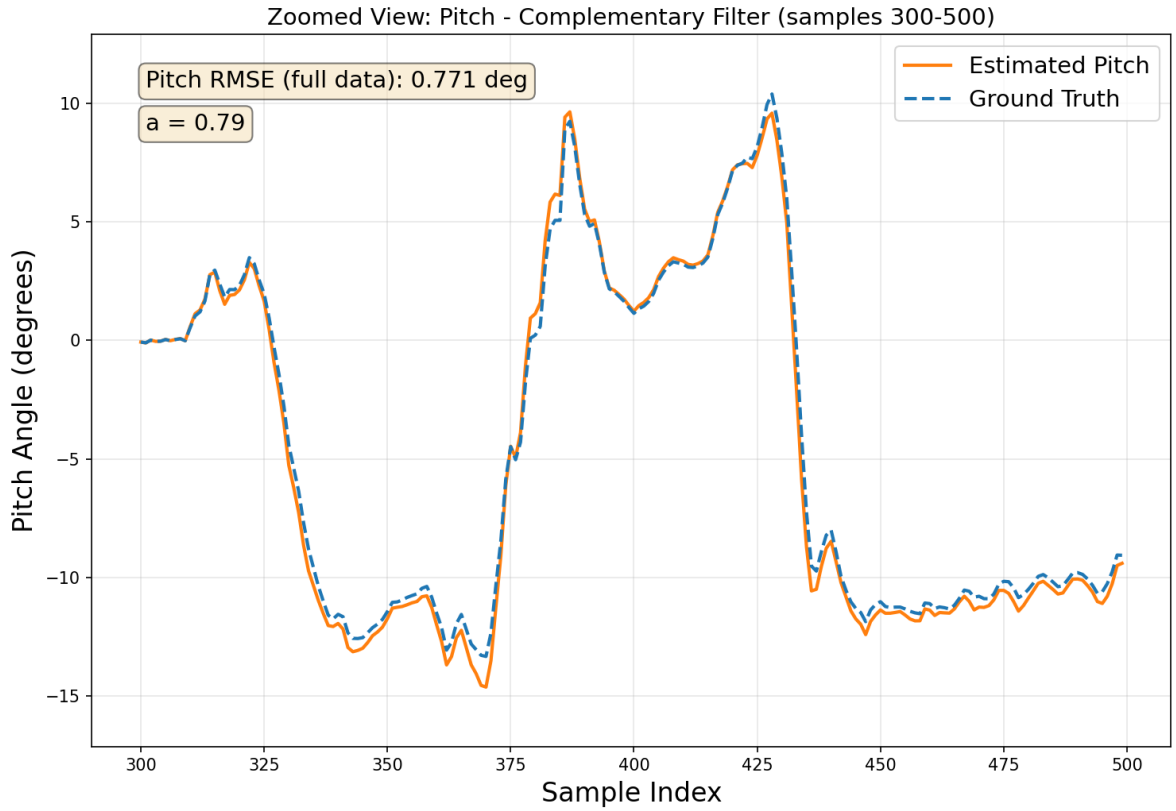



**Figure 2 Complementary Filter Zoomed view (samples 300-500) Pitch Estimation**

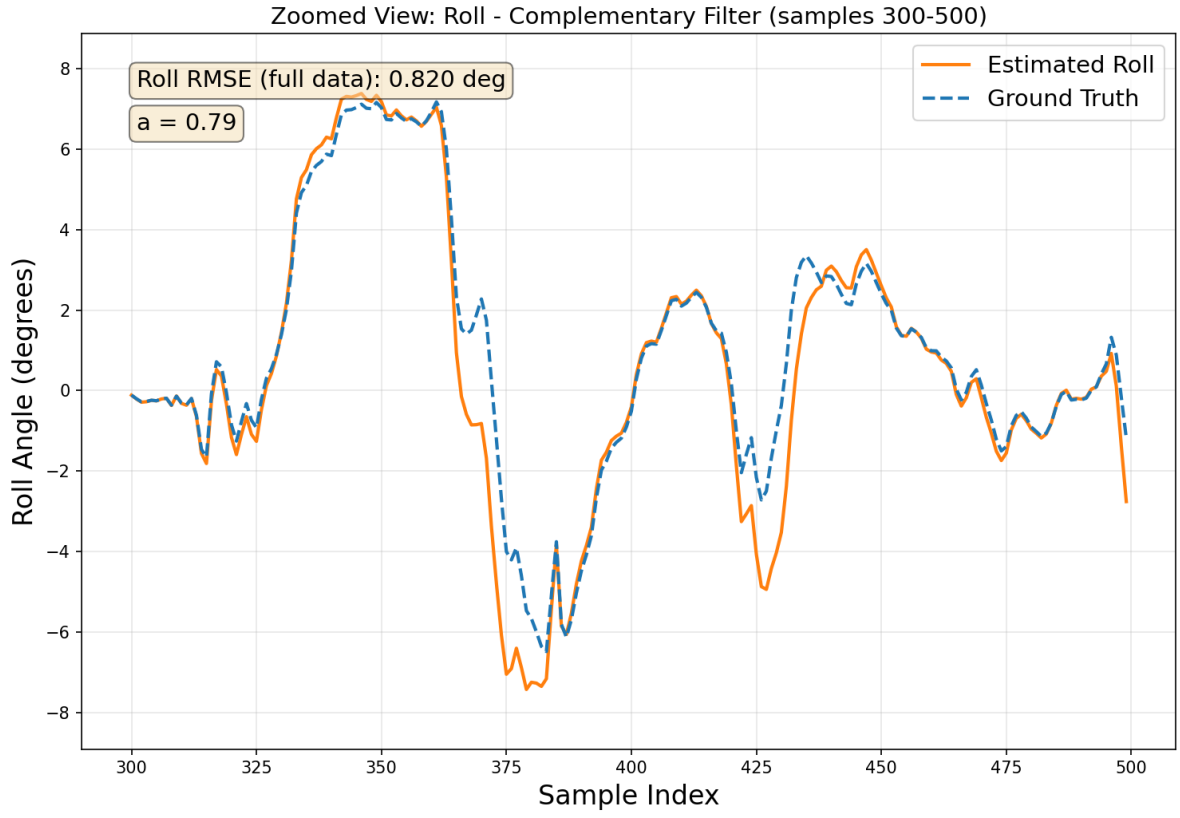



**Figure 3 Complementary Filter Zoomed view (samples 300-500) Roll Estimation**

Figure 2 zooms in the Pitch estimate during a dynamic maneuvering phase (samples 300-500). It can be observed that the Pitch still closely tracks the ground truth data. Figure 3 shows the Roll estimation for the same interval. Generally, it performs almost as well as pitch, with a minor exception around sample 375 where the filter momentarily fails to follow the truth data.

In summary, the Complementary Filter is an inexpensive, easy to implement, and effective solution for generic UAV attitude estimation. It removes most of the gyroscope drift and produces a smooth estimate that the flight controller can utilize. Its only downside is the static $\alpha$ gain. As $\alpha$ never varies, the filter is unable to adapt to the varying conditions encountered in practice.

## 3.3 Mahony Filter

The Mahony filter is a nonlinear complementary filter which operates directly on the SO(3) Special Orthogonal group [22]. The filter represents attitude as rotation matrices [22]. The main idea is to estimate a rotation matrix $\hat{R}$ by comparing the measured direction of gravity (accelerometer) with the estimated direction of gravity. This error between the two vectors will force the gyroscope to integrate towards its true vertical. The below implementation is a "Passive" Mahony filter as described in [22].

### 3.3.1 Algorithm Overview

The Mahony filter keeps an estimated rotation matrix $\hat{R}$ which represents the current attitude. At each timestep the filter calculates the difference between the measured direction of gravity (from the accelerometer) and the direction of gravity predicted by the current estimate. The error between these two is used to compute a correction term that rotates the gyroscope integration towards the true orientation. A rotation matrix $R_y$ is built from the accelerometer tilt angles, and the attitude error is computed as:

$$\tilde{R} = \hat{R}^T R_y \tag{16}$$

The correction vector is extracted from the anti-symmetric part of this error matrix:

$$\omega_{mes} = vec\left(P_a(\tilde{R})\right), P_a(\tilde{R}) = \frac{1}{2}(\tilde{R} - \tilde{R}^T) \tag{17}$$

The rotation matrix is then updated using the corrected angular velocity:

$$\dot{\hat{R}} = \hat{R}\left[\omega_y + k_p \omega_{mes}\right]_\times \tag{18}$$

After each integration step, $\hat{R}$ is projected back onto $SO(3)$ via SVD orthonormalization to prevent drift.

### 3.3.2 Parameters

This particular implementation of Mahony filter has just one parameter, the proportional gain $k_p$. It controls the accelerometer-based correction applied to the gyroscope integration at each timestep. A small $k_p$ (close to zero) allows the filter to depend primarily on the gyroscope for

attitude estimation. A large $k_p$ applies aggressive corrections to the gyroscope output based on the accelerometer measurement. This removes gyroscope drift, but makes the estimate more susceptible to accelerometer noise and to non-gravitational acceleration that occur during maneuvers [22].

The optimal $k_p$ needs to meet both of these conditions. It must be large enough that gyro drift is kept in a reasonable range during the operating period of interest, but small enough that estimates remain smooth during dynamic maneuvers. Unlike the Complementary Filter's a parameter which directly weights two angle estimates, $k_p$ works in the angular velocity domain, scaling the correction signal that is added to the gyroscope before integration.

To determine the optimal value, a grid search was performed over $k_p \in [1,100]$ with integer steps. For each candidate value, the Mahony filter was executed on the full dataset and the RMSE of both $\phi$ and $\theta$ was computed against ground truth. The optimization criterion was the average of the roll and pitch RMSE, ensuring that both angles are estimated well simultaneously. The search identified $k_p = 11$ as the optimal value.

### 3.3.3 Experimental Results

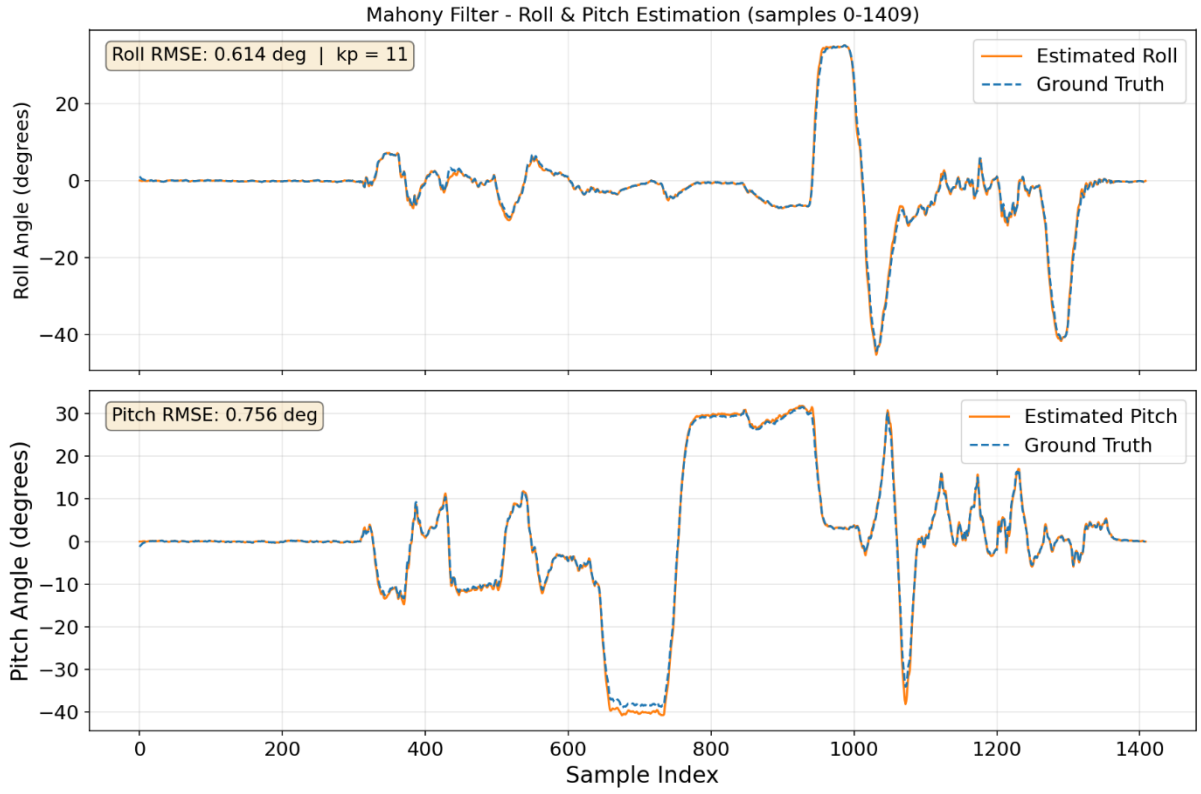



**Figure 4 Mahony Filter Roll and Pitch Estimation**

Figure 4 represents the global performance of the Mahony filter over the entire flight duration (1409) samples. The estimated $\phi$ and $\theta$ angles show a strong correlation with the ground truth data. The filter achieves an RMSE of 0.614 for $\phi$ and 0.756 for $\theta$, representing a significant

improvement over the Complementary Filter. The improvement is particularly notable for the roll angle $\phi$, where the filter reduces the RMSE by approximately 25%.

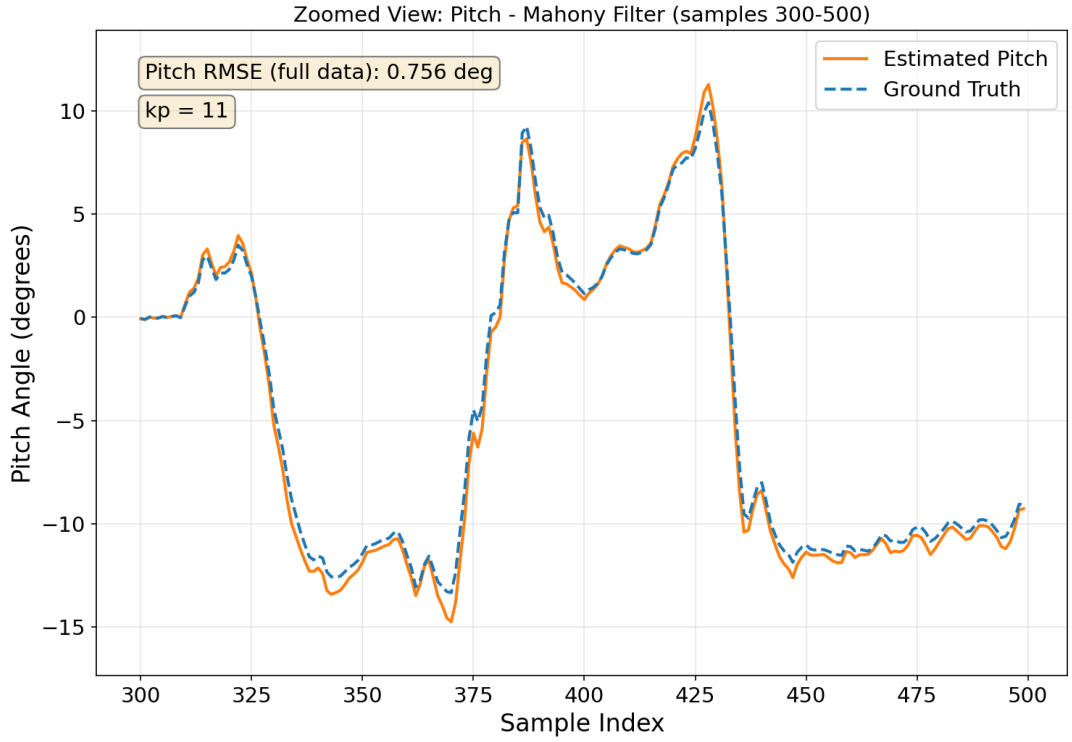



**Figure 5 Mahony Filter Zoomed view (samples 300-500) Pitch Estimation**

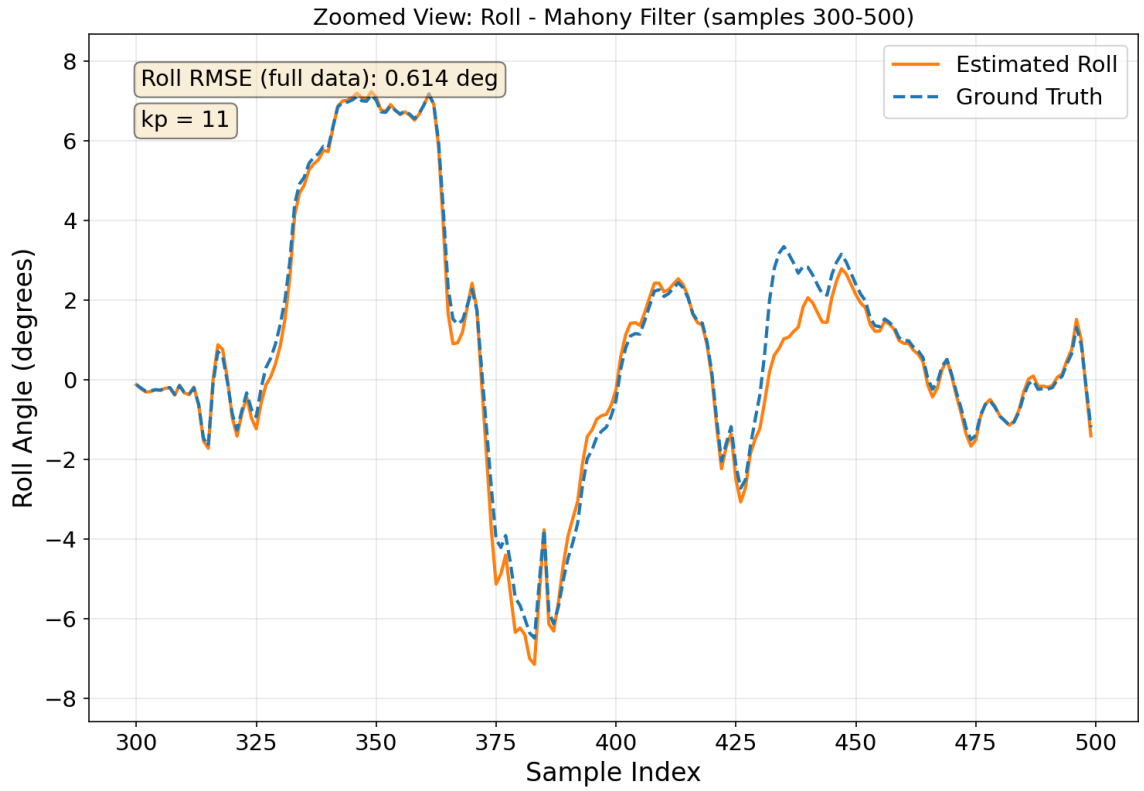



**Figure 6 Mahony Filter Zoomed view (samples 300-500) Roll Estimation**

Figure 5 and Figure 6 provide zoomed view of the attitude angles during a maneuvering phase (samples 300-500). Compared to the Complementary Filter, the Mahony filter tracks the rapid angles changes more closely. The geometric error correction on $SO(3)$ allows the filter to handle large rotation more gracefully than the Euler angle-based approach of the Complementary Filter.

The Mahony Passive Complementary Filter estimates attitude more accurately than the basic Complementary Filter, and it is still light enough to run in real time. Operating on $SO(3)$ with rotation matrices eliminates gimbal lock and allows geometric error correction. The single hyperparameter $k_p$ makes tuning easy, and the SVD-based orthonormalization ensures numerical stability during long runs. However, the proportional-only correction means that persistent biases in the gyroscope may not be fully eliminated over very long time periods, a limitation that is addressed by the Explicit Complementary Filter in the next section.

## 3.4 Explicit Complementary Filtering

The Explicit Complementary Filter extends the Mahony approach by introducing two important modifications. Firstly, it operates directly on sensor measurements without requiring intermediate attitude reconstruction. Secondly it incorporates integral feedback for online gyroscope bias estimation. Both features originate from Section V of Mahony et al. [22], where the filter is derived as an "explicit" complement to the passive formulation presented in the previous section.

### 3.4.1 Algorithm Overview

The filter maintains two state variables, an estimated rotation matrix $\hat{R}$ in $SO(3)$ and an estimated gyroscope bias vector $\hat{b}$ in $R^3$. At each timestep, the correction term is computed as the cross product between the measured and estimated gravity directions. The cross product produces a correction vector that is zero when the two directions are aligned, and grows proportionally to the angular error between them. The core filter equations are:

$$\dot{\hat{R}} = \hat{R}\left[\omega^y - \hat{b} + k_p\omega_{mes}\right]_\times \tag{19}$$

$$\dot{\hat{b}} = -k_i\omega_{mes} \tag{20}$$

where:
$\hat{b}$: The estimated gyroscope bias vector
$k_i$: The internal gain for bias correction
$\omega_y$: The raw gyroscope measurement

The first equation drives the rotation matrix forward. It uses the raw gyro data $\omega^y$ corrects it using the current bias estimate $\hat{b}$, and injects a proportional feedback term scaled by $k_p$. Meanwhile, the second equation uses integral feedback (via the $k_i$ gain) to continuously push the bias estimate toward the hardware's actual bias estimate. Because integration always introduces some level of numerical drift, the system finishes each discrete step by running SVD orthonormalization to project $\hat{R}$ safely back into a valid $SO(3)$ state.

### 3.4.2 Parameters

The filter has two tunable hyperparameters the proportional gain $k_p$ and the integral gain $k_i$. The first controls the correction applied to the gyroscope integration at each timestep, serving the same role as it does in passive Mahony filter. The latter dictates the adaption speed of the bias estimator. When $k_p$ is small the filter relies primarily on the gyroscope integration. On the other side when $k_i$ is small it prioritizes estimation stability and noise rejection.

To determine the optimal values, a two-dimensional grid search was performed over $k_p$ in $[1.0, 15.0]$ with a step size of 0.5 and $k_i$ in $[0.05, 1.0]$ with a step size of 0.05, resulting in 580 parameter pairs. For each pair, the filter was executed on the full dataset and the average RMSE of $\phi$ and $\theta$ was computed against ground truth. The grid search identified $k_p = 11$ and $k_i = 0.05$ as the optimal combination.

### 3.4.3 Experimental Results

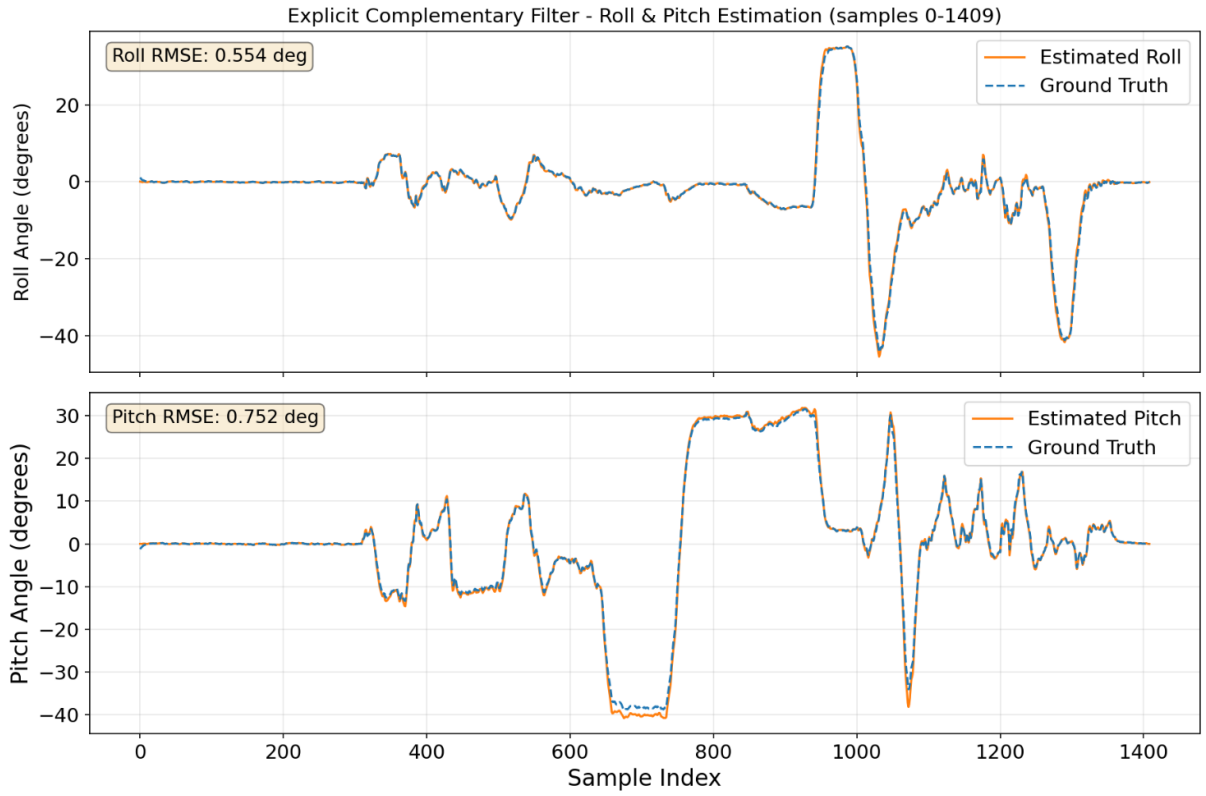



**Figure 7 Explicit Complementary Filter Roll and Pitch Estimation**

Figure 7 represents the global performance of the Explicit Complementary Filter over the entire flight duration. It achieves an RMSE of 0.554 degrees for $\phi$ and 0.752 degrees for $\theta$. Comparing it with the Passive Mahony Filter the roll RMSE is reduced by 10% and it still maintains a good enough pitch performance.

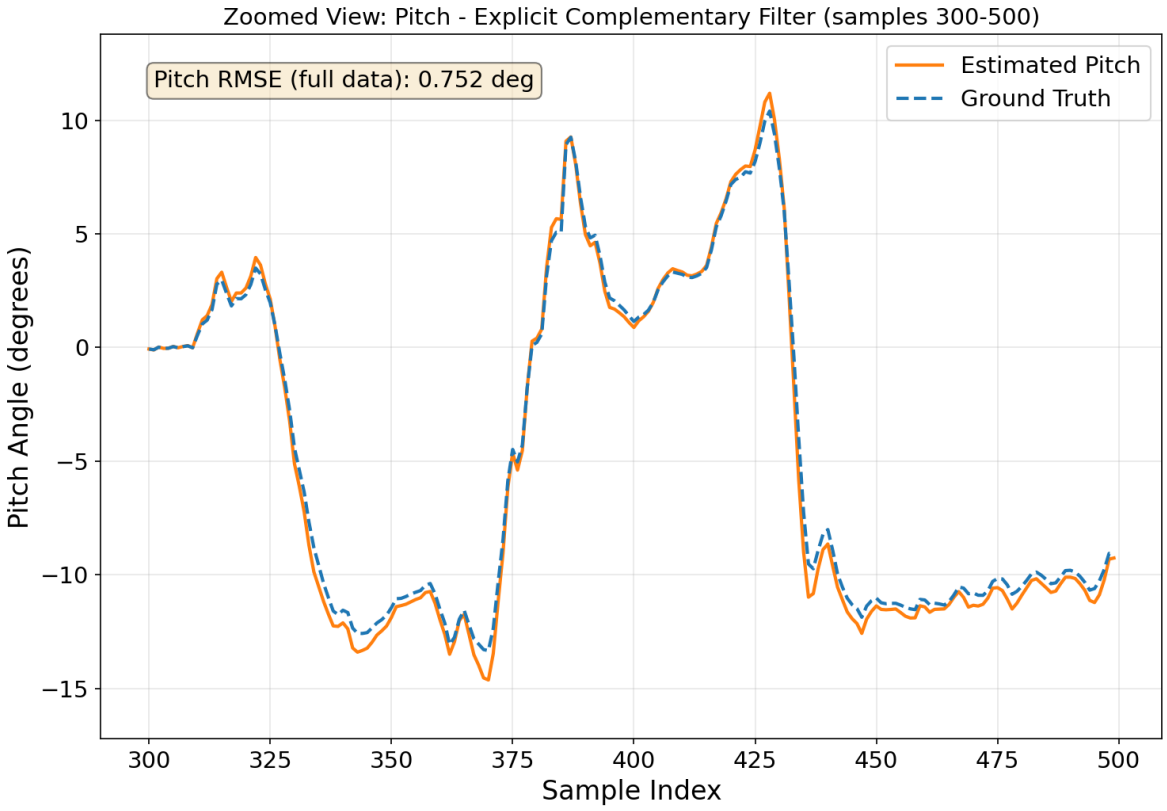



**Figure 8 Explicit Complementary Zoomed view (samples 300-500) Pitch Estimation**

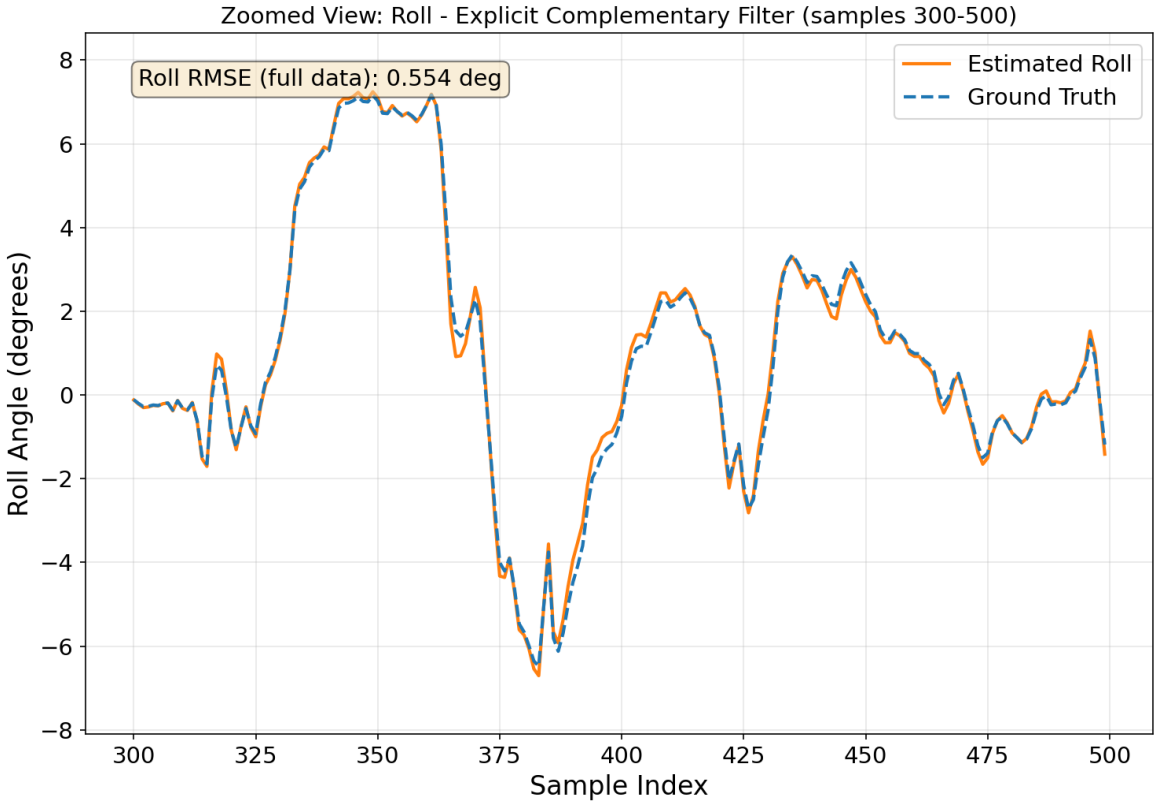



**Figure 9 Explicit Complementary Zoomed view (samples 300-500) Roll Estimation**

Figure 8 and Figure 9 zoom into the attitude angles during a dynamic maneuvering phase. Predictions are closely following the ground truth. The improvement is mostly noticeable in the roll angle.

In conclusion, the Explicit Complementary Filter demonstrates a modest improvement over the Passive Mahony Filter. Roll estimation accuracy is improved 10%. Both of the filters share the same proportional correction mechanism that operates in the $SO(3)$. They also share the same SVD-based orthonormalization, and the same optimal $k_p$ value. Since the filter operations directly on vector measurements, it has a computational advantage and avoids trigonometric mathematics. This allows it to run on resource-constrained embedded platforms.

## 3.5 Extended Kalman Filtering (EKF)

The Extended Kalman Filter is a probabilistic state estimation framework that extends the classical Kalman filter to nonlinear systems through the first-order linearization [23]. Unlike the complementary filters presented in the previous sections, the EKF maintains an explicit uncertainty estimate in the form of a covariance matrix, and fuses sensor measurements in a statistically optimal manner under the assumption of Gaussian noise.

The EKF represents attitude using quaternions. Quaternions avoid the singularities inherent to Euler angle representations (gimbal lock) and are computationally more efficient than rotation matrices for integration [24]

### 3.5.1 Algorithm Overview

The EKF operates through a recursive Predict-Correct cycle. It maintains a state vector $x$, which includes the attitude quaternion and gyroscope biases. The algorithm is divided into two main stages:

***Prediction (Time Update)***

In this stage, the filter projects the current state and uncertainty forward in time using the gyroscope measurements $\omega_y$

- State Propagation: The quaternion is updated using first-order kinematics:

$$q_{k+1} = q_k + \frac{\Delta t}{2}\Omega(w)q_k \tag{21}$$

  where $w = \omega_y - b$ represents the bias-corrected angular velocity.
- Covariance Extrapolation: The uncertainty (covariance matrix P) is projected forward:

$$P_{k+1} = FP_kF^T + Q \tag{22}$$

***Correction (Measurement Update)***

Once an accelerometer measurement $a_{norm}$ is received the filter "corrects" its prediction by comparing it to the expected gravity vector.

- Innovation: The filter calculates the difference (innovation y) between the measured acceleration and the predicted gravity direction

$$y = a_{norm} - h \tag{23}$$

- Kalman Gain: A weighting factor K is computed to determine how much to trust the new measurement versus the internal prediction:

$$K = PH^T(HPH^T + R)^{-1} \tag{24}$$

- State Update: The final state and covariance are updated:

$$x \leftarrow x + Ky$$
$$P \leftarrow (I - KH)P \tag{25}$$

where:
$x$: The 7-dimensional state vector $[q, b]^T$
$F, H$: The Jacobians representing the linearized system and measurement models.
$Q, R$: The process and measure noise covariance matrices.
$g = [0,0,1]^T$: The normalized gravity vector.

### 3.5.2 Parameters

The EKF is determined by two noise covariance matrices, the process noise Q (7x7) and the measurement noise R (3x3) [25]. They represent the degree of the filter's belief in the process model and the sensor measurements, respectively.

The process noise Q sets the level of uncertainty added during each prediction step. A large Q makes the filter trust less in the propagation and more in the measurements, while a small Q makes the filter trust more in the gyroscope integration [25]. Different values are set for the quaternion components and the bias states, since they evolve at different timescales.

The measurement noise R represents the a priori expected noise level of the accelerometer. A large R will make the filter reject accelerometer corrections, which is correct during a dynamic maneuver with significant non-gravitational accelerations present in the measurement. A small R will make the filter more responsive to accelerometer corrections at the expense of noise sensitivity.

The values selected for this implementation were: $Q_q = 0.001 * I_4$ (quaternion process noise), $Q_b = 0.0001 * I_3$ (bias process noise) and $R = 0.1 * I_3$ (accelerometer measurement noise). These are typical values for quaternion-based EKF attitude estimation [25].

### 3.5.3 Experimental Results

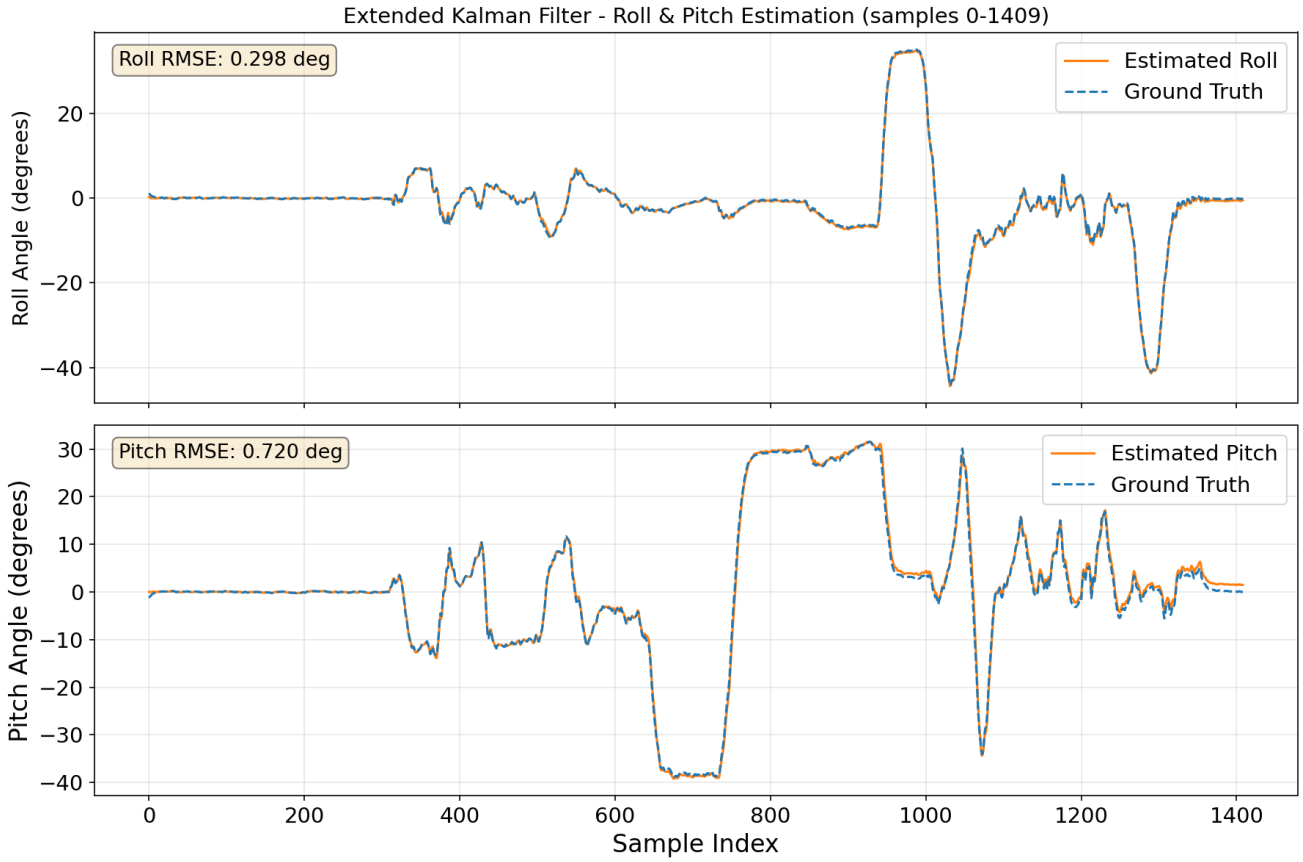



**Figure 10 EKF Roll and Pitch Estimation**

Figure 10 represents the global performance of the EKF over the entire flight duration. The filter achieves an RMSE of 0.298 degrees for $\phi$ and 0.720 for $\theta$. The roll estimation represents the best performance among all four filters implemented in this chapter, with a ~64% reduction in roll RMSE compared to the basic Complementary Filter and a ~51% reduction compared to Mahony filter.

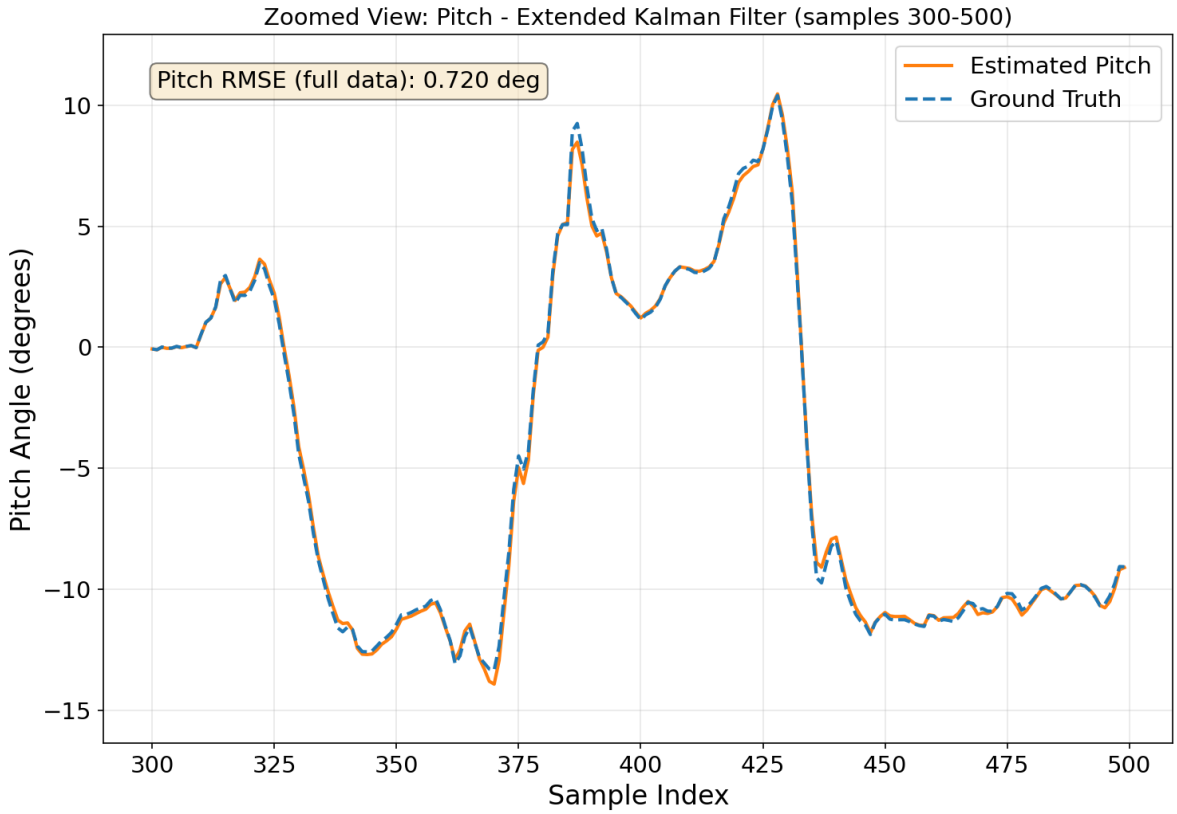



**Figure 11 EKF Zoomed view (samples 300-500) Pitch Estimation**

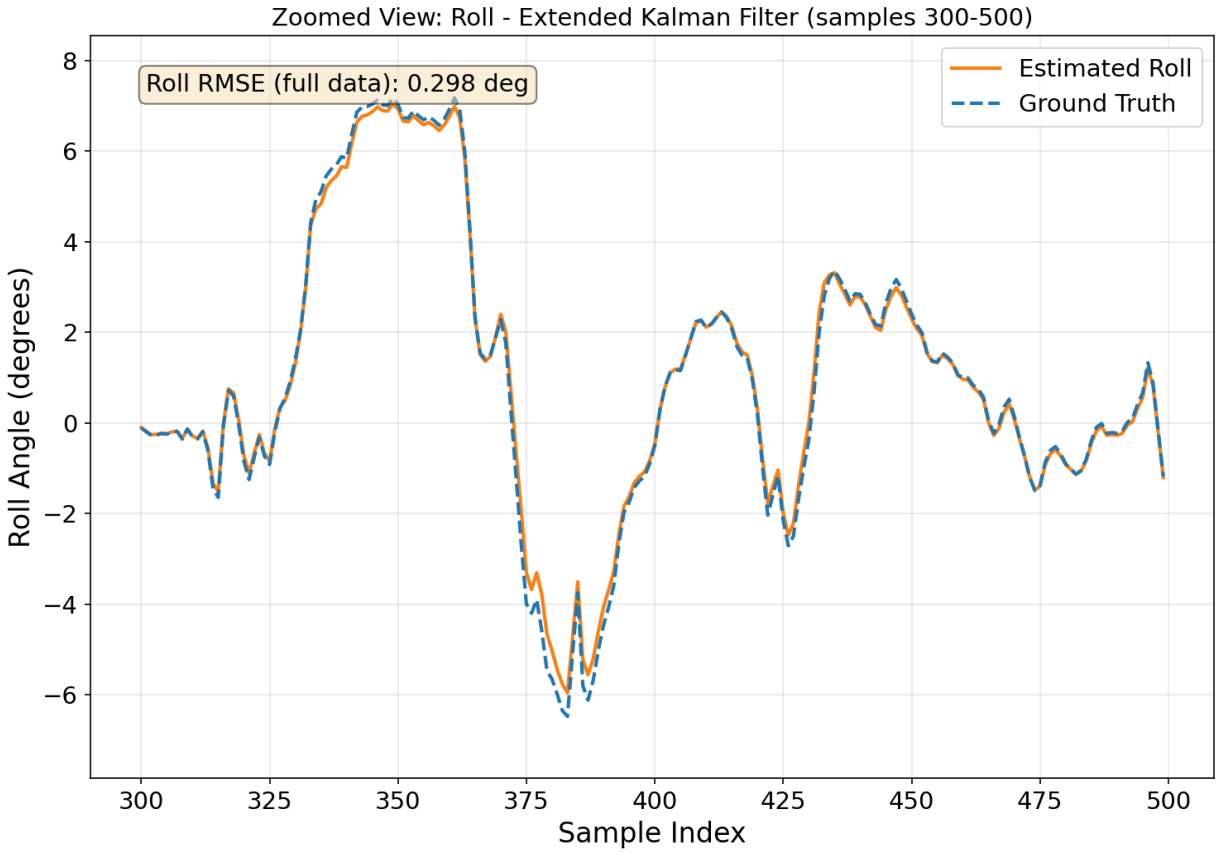



**Figure 12 EKF Zoomed view (samples 300-500) Roll Estimation**

Figure 11 and Figure 12 provide a zoomed view during the dynamic maneuvering phase (samples 300-500). The EKF tracks the ground truth more closely than the complementary filters, particularly in the roll axis where the bias estimation and probabilistic weighting provide the most benefit.

In conclusion, the EKF achieves the highest roll estimation accuracy of the four filters, demonstrating the advantage of probabilistic state estimation with explicit uncertainty tracking. However, this performance comes at a higher computational cost: at each timestep the filter maintains and updates a 7x7 covariance matrix and computes a 3x7 Kalman gain, making it more demanding than the complementary filter approaches. Also, the tuning of the Q and R matrices is a harder task and requires deeper knowledge of the algorithm compared to the single variable tuning that was done in the Mahony and Explicit CF.

## 3.6 Limitations and Conclusions

Table 1 shows the performance of the four classical attitude estimation filters when applied to the flight dataset. For each approach, the Roll and Pitch RMSE values are reported, along with the main features of the associated algorithm.

The Extended Kalman Filter resulted in the highest global estimation accuracy among the tested methods with a Roll RMSE of 0.298° and a Pitch RMSE of 0.720°. Its overall performance, especially regarding the estimation of roll, showed that the probabilistic state estimation framework in combination with gyroscope bias correction and uncertainty modelling lead to significantly lower errors than the ones achieved with the complementary filtering approaches.

This observation is also in agreement with the quantitative results. A qualitative evaluation of the different filters can be gained from Figure 13 where the outputs of all four filters are plotted versus the ground-truth attitude over a dynamic maneuvering segment of the flight (samples 300–500). As can be seen over this interval, the Extended Kalman Filter tracks the reference more closely than any other method.

| **Filter Algorithm** | **Roll RMSE (degrees)** | **Pitch RMSE (degrees)** | **Key Characteristics** |
|---|---|---|---|
| Complementary | 0.820 | 0.771 | Computationally lightweight; Euler-based; static trust gain. |
| Mahony (Passive) | 0.614 | 0.756 | Operates on $SO(3)$. Geometric error correction; single gain ($k_p$) |

| | | | |
|---|---|---|---|
| Explicit CF | 0.554 | 0.752 | Direct vector measurements; adds integral gain ($k_i$) for bias. |
| Extended Kalman Filter (EKF) | **0.298** | **0.720** | Full probabilistic state estimation. |

**Table 1 Summary of Classical Filter Performance**

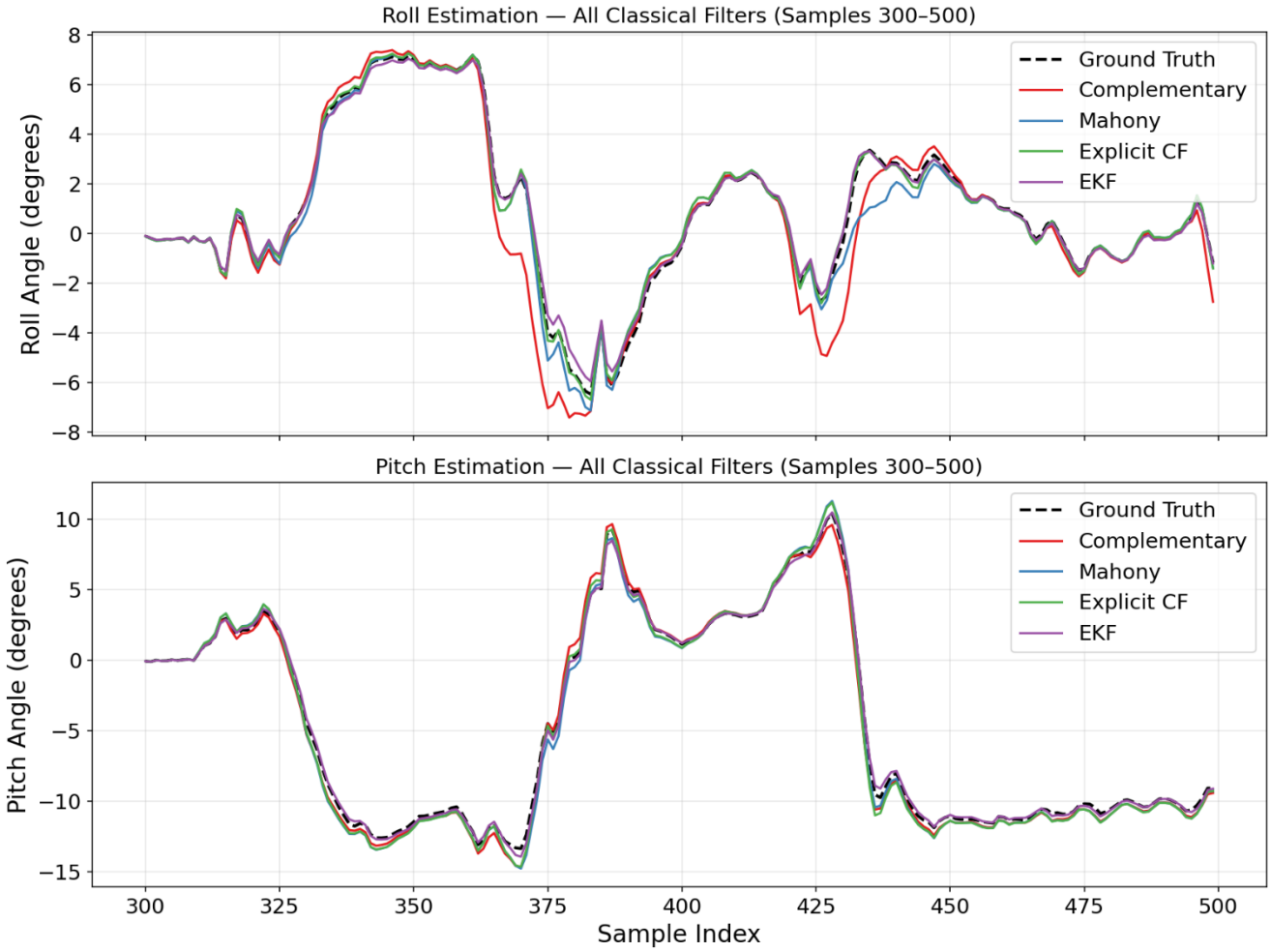



**Figure 13 Filters comparison in specific window**

# 4 Chapter 4: Deep Learning Approach

As seen in the previous chapter, standard tilt estimation filters are highly effective at estimating the UAV's current orientation. On our dataset, the EKF achieved a roll RMSE of 0.298° and a pitch RMSE of 0.720°, which essentially represents the upper performance limit for reactive estimation. But also, this is the core issue of the classical filters, they are fundamentally reactive. They rely on processing incoming sensor data which represent only the current state, meaning that they cannot predict attitude for the future at all. To fix this

weakness, this chapter shifts the focus from estimation to prediction. By feeding a look-back window of recent observations into an attention-based Long Short-Term Memory (LSTM) network, the goal is to predict the future attitude angles $(\phi, \theta, \psi)$ across $N$ upcoming timesteps. For modern proactive systems like MPC-based flight controllers, having these kinds of predictions is essential and mandatory.

## 4.1 Problem Formulation

Before diving into the depths of model architecture, training and evaluating we needed to have formal definition of what the model is trying to solve. This section gives a prediction task definition, shows the format and dimensions of input and output data and gives a quick overview of the training dataset.

### 4.1.1 Prediction Task Definition

The objective is to learn a mapping $f$ that given a sequence of $K$ consecutive observations vectors, predicts the attitude angles over the next $N$ timesteps. Formally, let $x(t) \epsilon R^d$ denote the observation vector at timestep $t$, where $d$ is the number of input features. The prediction tasks are:

$$f(x(k-K+1), \dots., x(k) = \big(\hat{y}(k+1), \hat{y}(k+2), \dots, \hat{y}(k+N)\big) \qquad (26)$$

where $\hat{y}(t) = \big[\hat{\phi}(t), \hat{\theta}(t), \hat{\psi}(t)\big]^T \epsilon\, R^3$ is the predicted attitude vector at future timestep $t$. In this formulation, the input windows spans timesteps $k-K+1$ through the current time k (the look-back window), while the output sequence spans timesteps $k+1$ through $k+N$ (the prediction horizon). Both $K$ and $N$ are configurable hyperparameters. For a given sampling rate period $T_s$, a look-back window of $K$ timesteps represents $K \cdot T_s$ seconds of flight history, while a prediction horizon of $N$ timesteps forecasts the vehicle's state $N \cdot T_s$ seconds into the future. The influence of various $K$ and $N$ configurations on prediction accuracy is evaluated experimentally in Section 4.4.

### 4.1.2 Input Feature Vector

Each input feature vector $x_t \in R^9$ needs to provide all the necessary details and UAV state in order for the model to make an accurate prediction.

Firstly, the current attitude angle estimation $(\phi, \theta, \psi)$ is needed. In a deployed system these values will be provided from a classic filter. As showcased before EKF has the advantage here. By integrating past attitude estimates it enables the model to internalize the temporal evolution of the vehicle's orientation, providing the necessary context for forecasting future states. Another important factor is the gyroscope angular velocities $(\omega_\phi, \omega_\theta, \omega_\psi)$. These are the angular rates that are measured directly by the IMU's 3-axis gyroscope in rad/s. These values will serve as indicator of near-term orientation changes Lastly, the control torques $(\tau_\phi, \tau_\theta, \tau_\psi)$ also have an important role. These are provided by the PID flight controller. It will allow the network to understand if an angular acceleration is caused by external environmental conditions or it was driven by a user command.

The above data provide the network with a detailed representation of the vehicle's dynamics. The attitude angles describe the current state, the gyroscope rates describe the rate of change, and the control torques describe the forces initiating that change. Together, these features encode the position, velocity, and forcing terms of the attitude dynamics.

### 4.1.3 Output Representation

The network output is a matrix $\hat{Y} \in R^{N\times 3}$ containing the predicted angles for the $N$ timesteps of the prediction horizon:

$$\hat{Y} = \begin{bmatrix} \hat{\phi}(k+1) & \hat{\theta}(k+1) & \hat{\psi}(k+1) \\ \hat{\phi}(k+2) & \hat{\theta}(k+2) & \hat{\psi}(k+2) \\ \vdots & \vdots & \vdots \\ \hat{\phi}(k+N) & \hat{\theta}(k+N) & \hat{\psi}(k+N) \end{bmatrix} \tag{27}$$

The output dimensionality depends on the prediction horizon $N$ $X$ 3. The architecture employs a **Direct Multi-Output** strategy, where the fully connected layer generates all values simultaneously in a single forward pass.

### 4.1.4 Dataset

The LSTM network is trained on a dataset that was created from flight data over seven different flights. This strategy is fundamentally different from what was done in the Classical filters of Chapter 3. The individual flight recordings were merged into a unified dataset containing 21,164 samples recorded at a sampling frequency of $66.6\,Hz$ ($T_s = 0.015s$), corresponding approximately to 317 seconds (~5.3 minutes) of total flight time. Each sample is a 9-dimensional vector containing the three feature groups described in Section 4.1.2.

To use many flights is an intentional architectural choice. A model trained only on a single flight has a risk of learning specific patterns rather than general attitude dynamics. By combining many flights with different profiles, the dataset exposes the network to a wide range of operating conditions. Roll has a range of $[-24°, 13°]$, pitch $[-14°, 20°]$ and yaw $[-21°, 22°]$. This large diversity is to help the model to generalize rather than memorize.

## 4.2 Network Architecture

This section gives a detailed view of the input, output and the architecture of the attention-based LSTM network that was developed. The model is designed as a deep multi-stage pipeline that transitions from temporal feature encoding to dynamic importance weighting, and finally to multi-step regression. The complete architecture is shown in Figure 14 and summarized by the following processing chain:

$$\text{Input}(9) \rightarrow \text{LSTM}(128) \rightarrow \text{Dropout}(0.2) \rightarrow \text{Attention} \rightarrow \text{Dropout}(0.2) \rightarrow \text{FC}(N \times 3) \rightarrow \text{Reshape}$$

As shown in Figure 14, the architecture consists of the following key components:

1. **Temporal Encoding**: A core LSTM layer with 128 hidden units processes the 9-dimensional input data. The layer is responsible for capturing the non-linear dependencies between IMU measurements and control inputs.
2. **Regularization**: Two dropout layers ($p = 0.2$) exist to prevent overfitting. The first layer follows the LSTM to ensure the model does not rely on specific hidden units, while the second follows the attention mechanism to regularize the final mapping.
3. **Attention Mechanism**: It evaluates all hidden states in the look-back window. It utilizes a scoring function with a *tanh* activation and a *softmax* layer to produce a context vector that emphasizes the most critical timesteps for future predictions.
4. **Multi-Output Regression**: The attended context vector is mapped via a fully connected (FC) layer to a flattened vector of size $N \times 3$, which is subsequently reshaped to represent the predicted roll, pitch, and yaw angles over the $N$-step horizon.

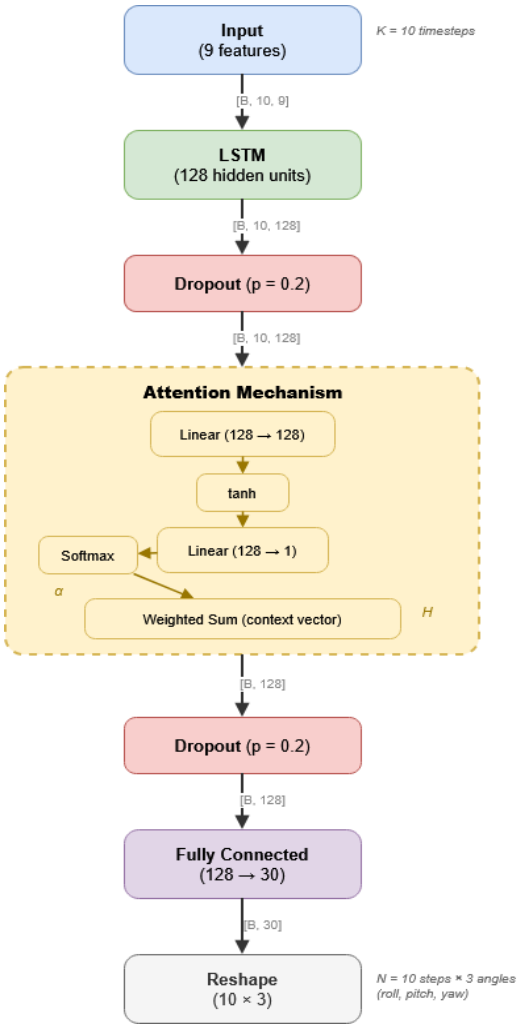



**Figure 14 LSTM Attention Based Network Architecture for K=10**

### 4.2.1 LSTM Layer

The first stage of the network is one LSTM layer that processes the input sequence and produces a hidden state representation at each timestep. As introduced in Section 2.4.3, the LSTM maintains a cell state $c_t$ and a hidden state $h_t$ that are updated at each timestep through the

forget, input, and output gates. This gating mechanism allows the network to selectively retain or discard information across timesteps, addressing the vanishing gradient problem that limits standard RNNs on sequences of this length.

The LSTM receives the input tensor $X \in R^{BxKxd}$, where $B$ is the batch size, $K$ is the look-back window length, and $d = 9$ is the number of input features. At each timestep $k$, the LSTM processes the feature vector $x_t \in R^9$ together with the previous hidden state $h_{t-1}$ and cell state $c_{t-1}$ producing an updated hidden state $h_t \in R^H$ where $H$ is the hidden state dimensionality. The output of the LSTM layer is the full sequence of hidden states: $H = [h_1, h_2, \dots, h_K], h_t \in R^H$

Unlike standard recurrent architectures that discard intermediate states and only utilize the final hidden state $h_K$ for prediction, this design retains the entire sequence $\boldsymbol{H}$. By preserving the hidden state at every timestep, the network maintains a dense temporal context that is required for the subsequent attention mechanism.

### 4.2.2 Attention Mechanism

The attention mechanism is the novel architectural feature that differentiates this network from a vanilla LSTM predictor. The role of the attention mechanism is to calculate a weighted sum of all hidden states $h_1, \dots, h_K$ so that the network can learn, at test time, how much of each timestep in the look-back window should be used to make the final prediction. The implementation uses an additive scoring function to compute a context vector $f(c) \in R^H$. An alignment score $s_t$ for each hidden state $h_t$ is computed using a learned non-linear projection:

$$s_t = v^T \tanh(W_a \cdot h_t + b_a) \tag{28}$$

where $W_a \in R^{H \times H} and\ v \in R^H$ are the weight matrices and vectors, respectively. These scores are normalized via a softmax operation to produce the attention weights $\alpha_t$:

$$\alpha_t = \frac{\exp(s_t)}{\sum_{j=1}^{K} \exp(s_j)} \tag{29}$$

The final attended representation is computed as the weighted sum of the hidden states: $c = \sum_{t=1}^{K} \alpha_t \cdot h_t$. This allows the network to prioritize informative timesteps regardless of their position in the look-back window, which is critical for capturing the varying dynamics of UAV flight.

### 4.2.3 Dropout Regularization

Training a neural network on a dataset of finite samples has a risk of overfitting, as the model memorizes specific patterns instead of generalizing. One of the most effective ways to handle this is to add regularization dropout [21] at two points in the forward pass:

1. **After the LSTM hidden states** - before the attention mechanism process them This prevents the attention layer from becoming very reliant on specific hidden units and encourages the LSTM to distribute information across the full hidden state.

2. **After the context vector** – before the fully connected output layer. This prevents the output layer from adapting to specific combinations of attended features.

Both dropout layers use a rate of $p = 0.2$, meaning that during each training forward pass, 20% of the values in the affected tensors are randomly set to zero and the remaining values are scaled by $\frac{1}{1-p}$ to maintain the expected magnitude. During evaluation dropout is disabled and all units are active ensuring deterministic predictions.

### 4.2.4 Output Layer

The context vector $c \in R^H$ is passed through a single fully connected layer that maps it to $N\ X\ 3$ output values, which are then reshaped into the prediction matrix $\hat{Y} \in R^{N\ X\ 3}$. No activation function is applied, as this is a regression task requiring unconstrained output values. A single output layer is sufficient because the attention mechanism already provides a rich, nonlinearly-processed representation of the input; additional depth would increase complexity without proportional benefit for the low output dimensionality.

## 4.3 Training Methodology

This section describes the complete training pipeline: how training samples are constructed from the raw time series, how the data is split for evaluation, the preprocessing applied, and the optimization configuration used to train the network.

### 4.3.1 Sliding Windows

Training samples are constructed using a sliding window approach over the concatenated time series. For each index $i$ (where $0 \leq i \leq S - K - N$ with $S = 21{,}164$, the total number of samples), a training pair $X_i, Y_i$ is constructed:

**Input:**$X_i \in R^{K \times d}$ - rows $i$ through $i + K - 1$ of the dataset, containing $K$ consecutive observation vectors with $d = 9$ features each.

**Target:**$Y_i \in R^{N \times 3}$ – rows $i + K$ through $i + K + N - 1$, containing only the attitude angle angles for the next $N$ timesteps.

With $S = 21{,}164$, this yields $S - K - N + 1$ sliding windows. The exact number depends on the chosen $K$ and $N$ values. The final tensor shapes are:

- Input tensor $\boldsymbol{X}$:[windows, $K$, 9]
- Target tensor $\boldsymbol{Y}$: [windows, $N$, 3]

Since the dataset is concatenated from 7 different flights, there will be a few windows that cross the gap between two flights. In practice, this only impacts at most 6 windows and has a negligible effect on training.

### 4.3.2 Data Standardization

Neural networks are sensitive to the scale of input features. In this dataset, the nine input features represent different physical quantities. Attitude angles are in radians, angular velocities in rad/s,

and control torques in Nm. To address these scale discrepancies, z-score standardization [26] is applied to the entire dataset. Each feature column is transformed to zero mean and unit variance using the column-wise mean and standard deviation. Because the network generates its predictions within this standardized space, all outputs are inverse-transformed back to radians prior to calculating the evaluation metrics, ensuring that all results reported in Section 4.4 are presented in their original physical scale.

### 4.3.3 Data Splitting

The combined flight data are split into three sets:

| Set | Proportion | Purpose |
|---|---|---|
| Training | 80% | To train the model |
| Validation | 10% | Monitor training progress and overfitting |
| Test | 10% | Performance evaluation |

**Table 2 Data split percentage**

Before performing the split, all of the windows are shuffled in a random order. This is required since the sliding windows are built by continuously stepping through the concatenated flights. If this shuffling was not performed, the test set would be made up entirely of windows from the last flight. The resulting validation and training sets would therefore both be representative of the same flight profile, and the ability of the model to generalize to other profiles would not be evaluated. The shuffling step ensures that each of the validation, training, and test sets include samples from all seven flights. During training, the validation loss is calculated every five epochs with dropout turned off. This is to allow for live tracking of training progress in terms of the model's ability to generalize, and also to help to detect overfitting. The test set is only used to calculate results once training is complete to ensure that the final results presented in Section 4.4 are calculated on **unseen** data.

### 4.3.4 Loss & Optimization

The network is trained to minimize Mean Squared Error (MSE) between predicted and target attitude angles. MSE penalizes large error quadratically, encouraging the network to avoid large deviations. The loss is computed on standardized values, ensuring all three angles contribute equally regardless of their original physical scales. The model is trained using the Adam optimizer [27] with the following configuration:

| Parameter | Value |
|---|---|
| Loss function | MSE |

| | |
|---|---|
| Optimizer | Adam |
| Learning rate | 0.001 |
| Batch size | 64 |
| Epochs | 500 |
| Dropout rate | 0.2 |

**Table 3 Training Configuration**

At each epoch, the training sample indices are shuffled and processed in mini-batches of 64. For each batch, a forward pass computes the predictions and loss, gradients are computed via backpropagation, and the optimizer updates the model parameters.

## 4.4 Experimental Results and Evaluation

This section presents the experimental evaluation of various LSTM architectures. To assess the impact of the attention mechanism, standard LSTM models without attention were explicitly trained and evaluated to serve as direct comparative baselines. In total, six model configurations are compared to evaluate the effects of the attention mechanism, the look-back window $K$ and prediction horizon $N$. All models share the same hidden state $H = 128$, dropout rate $p = 0.2$ and training configuration (Adam optimizer, $lr = 0.001$, batch size = 64 and 500 epochs).

| Model | Attention | K (look-back) | N (horizon) | Output size |
|---|---|---|---|---|
| **A** | Yes | 10 | 10 | 30 |
| **B** | No | 10 | 10 | 30 |
| **C** | Yes | 10 | 30 | 90 |
| **D** | No | 10 | 30 | 90 |
| **E** | Yes | 50 | 30 | 90 |
| **F** | No | 50 | 30 | 90 |

**Table 4 LSTM Model's Definition**

### 4.4.1 Overall Performance Comparison

Table 5 summarizes the test-set performance of all six model configurations across the three-evaluation metrics: overall Test RMSE, Test MAE, and per-axis RMSE for roll, pitch, and yaw. Each row corresponds to one configuration from Table 4, allowing a direct comparison of how

the attention mechanism, look-back window size K, and prediction horizon N jointly affect prediction accuracy.

| Model | Attention | K | N | Test RMSE | Test MAE | Roll RMSE | Pitch RMSE | Yaw RMSE |
|---|---|---|---|---|---|---|---|---|
| **A** | Yes | 10 | 10 | 0.437 | 0.282 | 0.524 | 0.357 | 0.414 |
| **B** | No | 10 | 10 | 0.444 | 0.302 | 0.527 | 0.357 | 0.431 |
| **C** | Yes | 10 | 30 | 0.521 | 0.372 | 0.613 | 0.440 | 0.494 |
| **D** | No | 10 | 30 | 0.502 | 0.360 | 0.577 | 0.435 | 0.484 |
| **E** | Yes | 50 | 30 | **0.420** | **0.289** | **0.472** | **0.315** | **0.455** |
| **F** | No | 50 | 30 | 0.440 | 0.310 | 0.480 | 0.356 | 0.474 |

**Table 5 LSTM Model's RMSE**

The benefit of attention is tied to the size of the look-back window. At $K = 10$, *attention provides negligible gains for short horizons* (Models A vs B) and actually degrades performance for long horizons (Models C vs. D: 0.521° vs 0.502°). The 10 available hidden states do not provide enough sequence variance for meaningful weighting. At $K = 50$ (Models E vs. F), attention results in large decrease in error (4.5% RMSE improvement, 6.8% MAE improvement). The wider window provides the attention layer with a sufficiently rich temporal landscape to recognize and leverage important past events.

Extending the prediction horizon from $N = 10$ to $N = 30$ without providing further context raises the error by ~13% (Models B vs D). However, increasing the look-back window from $K = 10$ to $K = 50$ can recover this loss (Models D vs F: 12.4% improvement). The combination of deep history and attention (Model E) represents the optimal configuration, achieving the lowest overall RMSE (0.420°) despite predicting three times further into the future than Model A.

Figure 15 provides the results as a group bar chart of the overall test RMSE for each configuration against its (N, K) setting. The blue bars represent the models that have an attention-based mechanism and the red bars the ones who do not have one. The chart clearly shows the interaction between the three design dimensions: Model E (N=30, K=50, attention) has the lowest overall RMSE of 0.420°, highlighted as the best configuration.

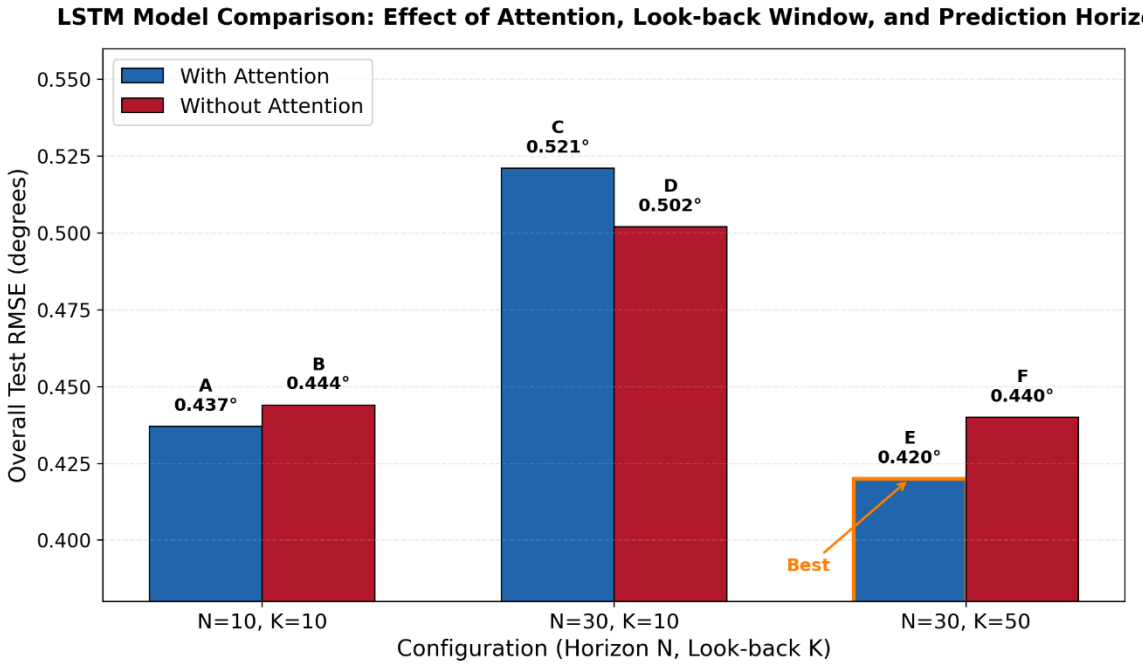



**Figure 15 LSTM Model Comparison Test RMSE**

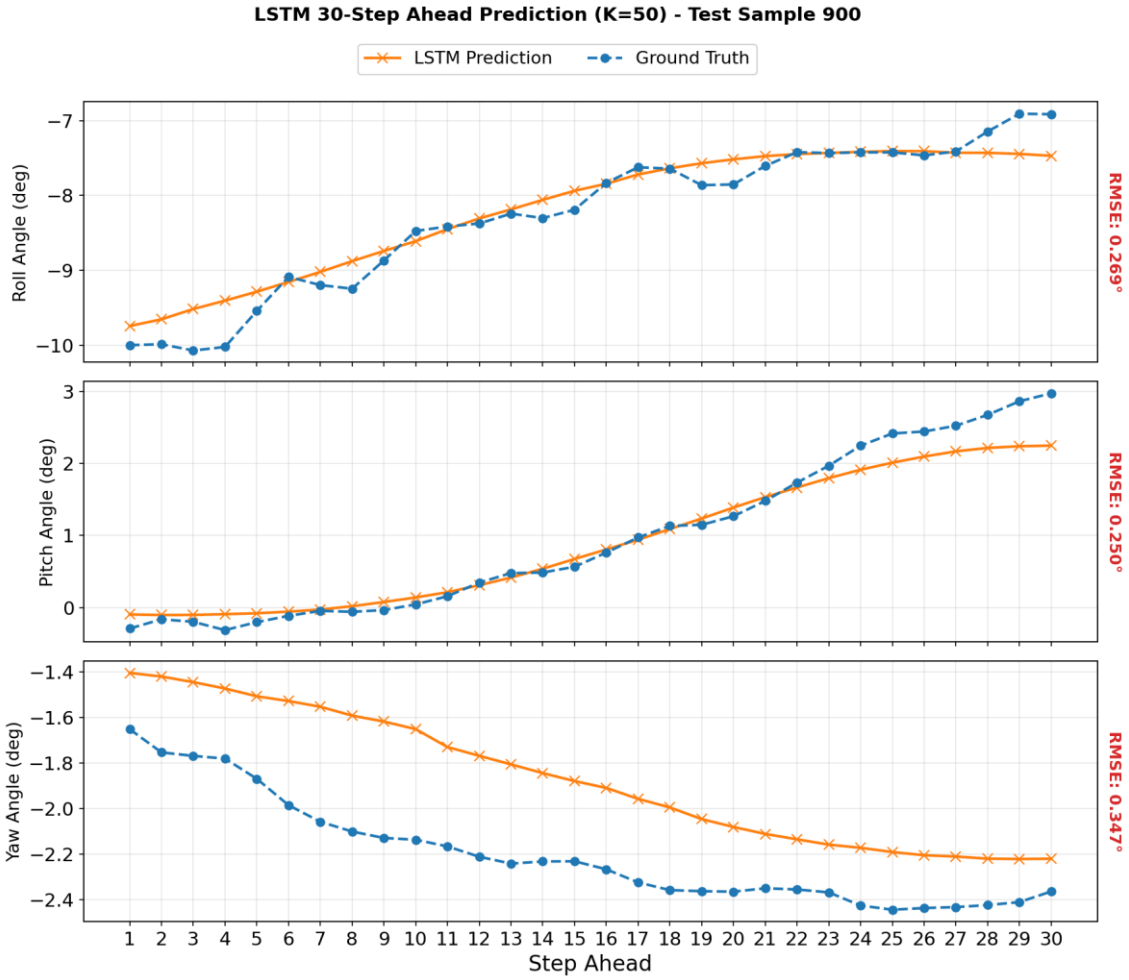



**Figure 16 Model E 30 steps prediction**

Figure 16 shows the forecasting performance of Model E across the full $N = 30$ prediction window. The plot demonstrates the network's capacity for roll and pitch to maintain consistent tracking and low prediction errors across both near-term and extended future horizons.

### 4.4.2 Error Growth Over the Prediction Horizon

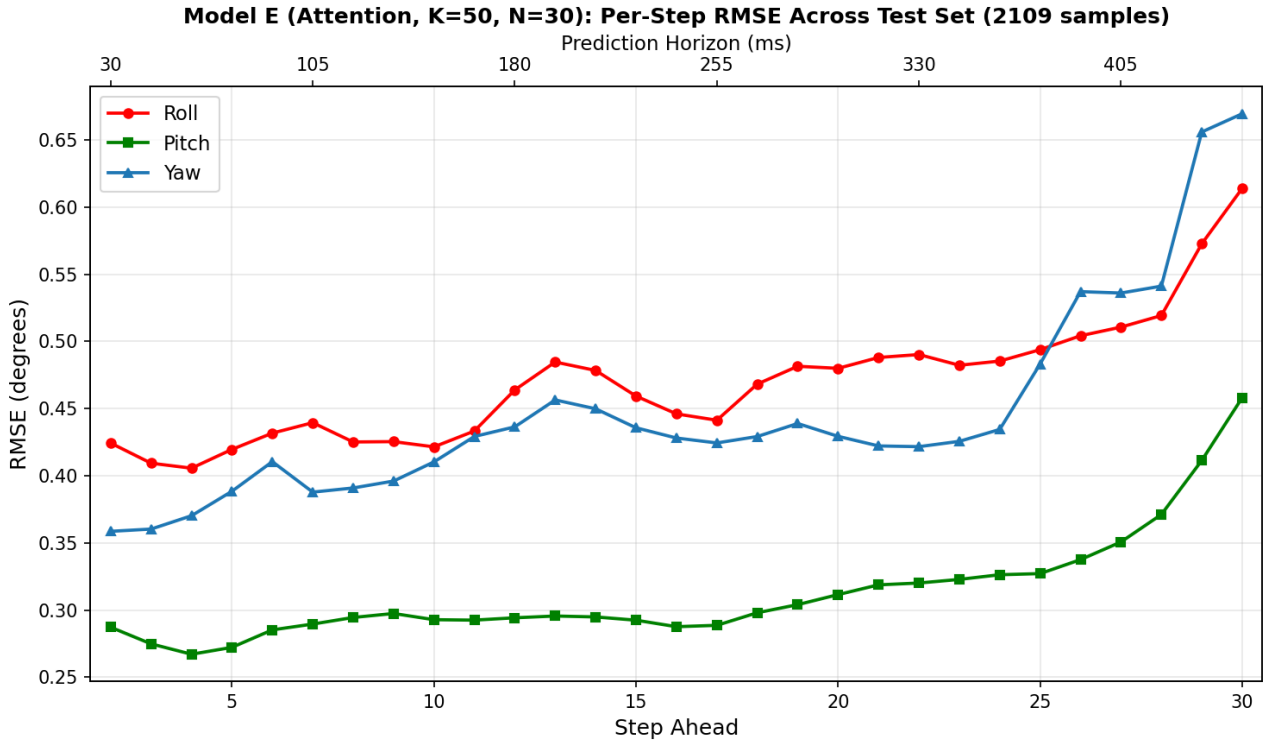



**Figure 17 Model E Error Growth**

To evaluate how the quality of the prediction deteriorates the more steps into the future are predicted, the per-step RMSE of Model E is shown in Figure 17, again evaluated on all 2,109 test samples. (In contrast to the performance numbers in Table 5, the RMSE is not averaged over all 30 steps here, so that the time structure of the error can be seen.) The error can be broken down into the three following parts:

**Steps 1–5 (0–75 ms)**: The model achieves its lowest errors in this near-term region, with pitch RMSE dropping to 0.267° at step 4 and roll stabilizing around 0.41°. Notably, step 1 shows a brief spike (particularly yaw at 0.50°) before the error drops at steps 2–3. This may indicate that the network's initial prediction is somewhat less certain (due to it being an initial prediction without ground-truth inputs), before this is rapidly corrected as the prediction trajectory pattern converges to the learned one.

**Steps 6–24 (75–360 ms):** The error remains remarkably stable across this extended mid-range region. Roll is between 0.42° and 0.49°, pitch between 0.27° and 0.33°, and yaw between 0.39° and 0.48°. This consistency indicates that the attention-based LSTM is able to maintain a certain level of prediction quality over a mid-term temporal horizon, spanning around 285 ms of future flight dynamics.

**Steps 25–30 (375–450 ms):** All three angles show a rapid increase in error. Roll rises from 0.49° to 0.61°, pitch from 0.33° to 0.46°, and yaw from 0.48° to 0.67°. This increase is the prediction limit of our model. It is clear enough that after ~375 ms the model can no longer predict accurately.

## 4.5 Discussion

### 4.5.1 The role of Attention

The experiments reveal one key fact about the attention mechanism. The mechanism is beneficial only when it has a big enough historical context. When the lookback window has 10 timestamps ($K = 10$) that represent only 0.15 seconds of flight, the network cannot use them effectively and assign the proper weights. When the look-back window was increased to keep 0.75 seconds ($K = 50$) of flight data the attention mechanism identified and prioritized past events that could lead to a sudden change of attitude.

### 4.5.2 Limitations

**Data Leakage from Overlapping Windows**: The sliding window construction produces extremely overlapping samples (K=50, 98% overlap between consecutive windows). Because windows are shuffled before being split into train/val/test sets, near-duplicate samples end up in different sets. The reported test metrics are thus optimistically biased. A better evaluation would use a flight-based cross-validation strategy (e.g. hold out whole flights for testing). Critically, since all six model configurations are tested using the same splitting protocol, the relative comparisons of architectures, attention effect, look-back window size and prediction horizon are totally unaffected. The bias does not affect the architectural conclusions.

**Global standardization statistics**: The z-score normalization that was applied to the network inputs used the mean and standard deviation of the full combined dataset (rather than being computed only from statistics from the training set). This is technically a very mild form of data leakage in the context of a machine learning pipeline. But since the training set is 80% of the full data, the global statistics are dominated by the training samples. The impact on the learned weights in the network and the corresponding prediction error is undetectably close to zero in practice.

**Limited flight diversity:** While the seven flights offer more diversity than a single recording, this dataset corresponds to one particular quadcopter platform, under laboratory conditions. It has not been tested with other UAVs, with other sensor configurations and in outdoor environments with wind disturbances (among other effects).

# 5 Conclusions

This thesis connected two mechanisms that are usually used separately in the UAV context, classical attitude estimation and deep learning prediction. In the first part four classical filter were implemented and compared. The EKF came out as a clear winner, showing the best performance for the specific training set. The second part was focused on the deep learning

architecture and how to use it to make multi-step ahead predictions. An attention based LSTM network was developed that was designed to run in parallel with the EKF filter.

After several experiments it was shown that the attention mechanism improves the prediction accuracy under a certain criterion. If the look-back window is short the attention mechanism does not have enough context to assign the proper weights, it was just noise for the network without any prediction accuracy improvement. But given a long enough window ($K = 50$) it had enough history to give an improved accuracy prediction. Across the six configurations tested, the balance between look-back length and prediction horizon turned out to be the deciding factor. The best one, Model E, reached an overall test RMSE of $0.420°$ while predicting 30 steps ahead.

Both the classical estimation filter and the LSTM network were written in C++ with the Eigen library usage for mathematical operations and LibTorch for the network. This allows the filters and network to run on the embedded hardware UAVs have.

# 6 Future Work

The model predicts accurately but there is still room for improvement. A first next step is to deploy the model into UAV hardware and do some real testing scenarios. That would show how well it handles data that was not captured. As stated in the Introduction, the multi-step ahead prediction is what an MPC needs. So, we can deploy it together with an MPC and measure how much the network predictions help to improve flight stability and control. Lastly the network needs to be explored more in order to check if it can be made small to run even on cheap, low-power hardware.

# Bibliography – References – Online sources

1. Protoulis, Teo & Alexandridis, Alex. (2025). Online Learning of a Real-World Quadcopter Attitude Dynamics using a Self-Organizing RBF Network. 1-8. 10.1109/IJCNN64981.2025.11227778.
2. Kapnopoulos, Aristotelis & Alexandridis, Alex. (2022). A cooperative particle swarm optimization approach for tuning an MPC-based quadrotor trajectory tracking scheme. Aerospace Science and Technology. 127. 107725. 10.1016/j.ast.2022.107725.
3. Protoulis, T., & Alexandridis, A. (2026). Navigation of multi-agent systems in cluttered and restricted environments using a distributed Lyapunov-based MPC framework. Systems & Control Letters, 213. https://doi.org/10.1016/j.sysconle.2026.106437
4. Protoulis, T., A. Alexandridis, “A Lyapunov-based Formation Control Framework for Multi-Agent Systems in Space-Restricted and Obstacle-Populated Environments”, 2026 American Control Conference (ACC), 2026, New Orleans, USA (accepted for presentation)
5. D. Vavelidou, T. Protoulis and A. Alexandridis, "Trajectory Tracking with Obstacle Avoidance for Autonomous UAV Swarms Based on Distributed Model Predictive

Control," 2024 International Conference on Unmanned Aircraft Systems (ICUAS), Chania - Crete, Greece, 2024, pp. 986-993, doi: 10.1109/ICUAS60882.2024.10556854

6. Kapnopoulos, A., C. Kazakidis, A. Alexandridis, "Quadrotor trajectory tracking based on backstepping control and radial basis function neural networks", Results in Control and Optimization, 14 (2024) 100335.
7. Zhou, X., Huang, J., Lu, F., Liu, J., & Wang, C. (2021). HNN-based generalized predictive control for turbofan engine direct performance optimization. Aerospace Science and Technology, 112, 106602. https://doi.org/10.1016/j.ast.2021.106602
8. Protoulis, Teo & Alexandridis, Alex. (2026). An optimal hierarchical control framework for navigation of fixed-wing UAV swarms in complex environments. Aerospace Science and Technology. 176. 112480. 10.1016/j.ast.2026.112480.
9. Liu, Y., Zhou, Y., & Li, X. (2018). Attitude Estimation of Unmanned Aerial Vehicle Based on LSTM Neural Network. 2018 International Joint Conference on Neural Networks (IJCNN), 1-6.
10. Weber, D., Guhmann, C., & Seel, T. (2021). "RIANN – A Robust Neural Network Outperforms Attitude Estimation Filters". AI (MDPI), 2(3), 444-463.
11. Golroudbari, A.A., & Sabour, M.H. (2023). Generalizable End-to-End Deep Learning Frameworks for Real-Time Attitude Estimation Using 6DoF Inertial Measurement Units. Measurement, 217, 113105.
12. Brossard, M., Bonnabel, S., & Barrau, A. (2020). Denoising IMU gyroscopes with deep learning for open-loop attitude estimation. IEEE Robotics and Automation Letters, 5(3), 4796-4803.
13. Cohen, N., & Klein, I. (2024). Inertial navigation meets deep learning: A survey of current trends and future directions. Results in Engineering, 24, Article 103565. https://doi.org/10.1016/j.rineng.2024.103565
14. Bianchi, D., Epicoco, N., Di Ferdinando, M., Di Gennaro, S., & Pepe, P. (2024). Physics-Informed Neural Networks for Unmanned Aerial Vehicle System Estimation. Drones, 8(12), 716. https://doi.org/10.3390/drones8120716
15. Zhou, X., Chen, L., Sun, C., Jia, W., Yi, N., & Sun, W. (2025). Highly Accurate Attitude Estimation of Unmanned Aerial Vehicle Payloads Using Low-Cost MEMS. Micromachines, 16(6), 632. https://doi.org/10.3390/mi16060632
16. Diebel, J. (2006). "Representing Attitude: Euler Angles, Unit Quaternions, and Rotation Vectors." Stanford University Technical Report.
17. Craig, J. J. (2005). Introduction to Robotics: Mechanics and Control. 3rd Edition. Pearson Prentice Hall.
18. Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. IEEE Transactions on Neural Networks, 5(2), 157-166.
19. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735-1780.
20. Higgins, W. T. (1975). A comparison of complementary and Kalman filtering. IEEE Transactions on Aerospace and Electronic Systems, AES-11(3), 321-325. https://doi.org/10.1109/TAES.1975.308081
21. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.

22. Mahony, R., Hamel, T., Pflimlin, J.-M. (2008). Nonlinear Complementary Filters on the Special Orthogonal Group. IEEE Transactions on Automatic Control, 53(5), 1203-1217. https://doi.org/10.1109/TAC.2008.923738
23. Welch, G., & Bishop, G. (1995). "An Introduction to the Kalman Filter." University of North Carolina at Chapel Hill, Technical Report TR 95-041.
24. Solà, J. (2017). Quaternion kinematics for the error-state Kalman filter (arXiv:1711.02508). arXiv. https://arxiv.org/abs/1711.02508
25. Sabatini, A. M. (2006). Quaternion-based extended Kalman filter for determining orientation by inertial and magnetic sensing. IEEE Transactions on Biomedical Engineering, 53(7), 1346-1356. https://doi.org/10.1109/TBME.2006.875664
26. LeCun, Y., Bottou, L., Orr, G.B., et al. (1998). Efficient Backprop. Neural Networks: Tricks of the Trade, 1524, 9-50. https://doi.org/10.1007/3-540-49430-8_2
27. Kingma, D. P., & Ba, J. (2015). "Adam: A Method for Stochastic Optimization", 3rd International Conference on Learning Representations (ICLR).